

INFO 490 MG:
A. I.
Web-Application Development

MOHIT GUPTA

Week 2:
Directory Structure + Github Hygiene +
Templates + Views & their Types

Index

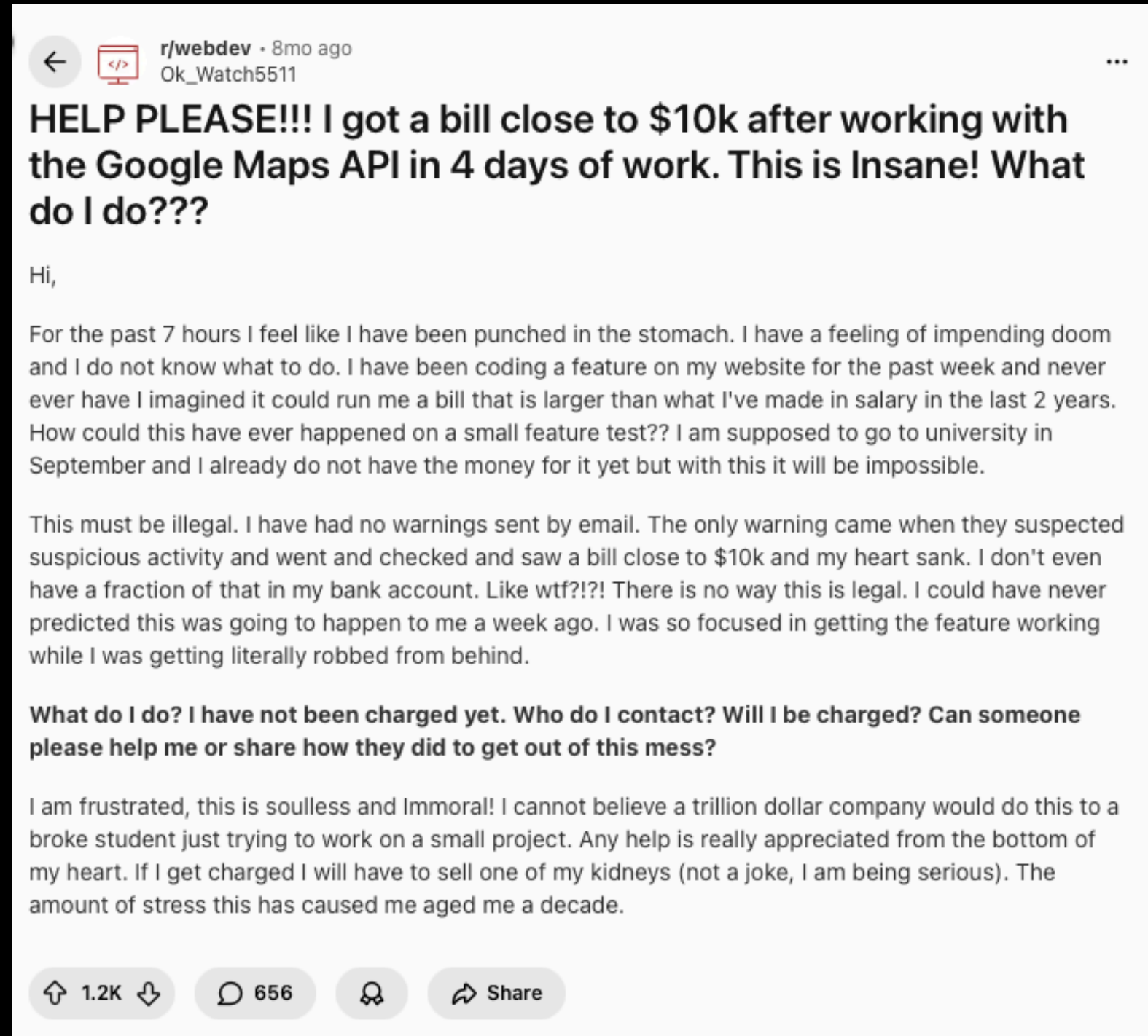
0. Revisiting Week-1	10:00 - 10:30 am (script-trial: 10:20 am)
1. Directory Restructuring.....	10:30 - 11:00 am (script-trial: 10:50 am)
2. .env (environment variables).....	11:00 - 11:30 am (script-trial: 11:20 am)
3. GitHub Hygiene.....	11:30 - 12:00 pm (script-trial: 11:50 am)
a. setup	
b. .gitignore	
c. secrets	
4. Templates.....	12:00 - 12:30 pm (script-trial: 12:20 pm)
5. Views.....	12:30 - 12:50 pm (script-trial: 12:40 pm)
a. Function Based Views	
b. Class Based Views	

0. Revisiting Week-1

- a. Differences in Virtual Environment
- b. Initiating Django Project
- c. Revisiting Demo Test Script
- d. Revisiting Models

Follow in-class demo.

What can be possible reasons for this bill?



https://www.reddit.com/r/webdev/comments/1kupy13/help_please_i_got_a_bill_close_to_10k_after/

Sort by: Best ▾

🔍 Search Comments



crybabe420 · 8mo ago

did you upload your api key to github or something? that's a lot



1.6K



Reply



Award



Share



[deleted] · 8mo ago



im_rite_ur_rong · 8mo ago

I bet this if the actual answer



435



Reply



Award



Share



```
1 .env
... @@ -0,0 +1 @@
1 + API_KEY=a54df9cd-a513-40cb-bffc-07351079651c
```

*developer



```
1 .env
... @@ -0,0 +1 @@
1 + API_KEY=a54df9cd-a513-40cb-bffc-07351079651c
```

*manager



<https://youtube.com/shorts/zgR4GmrcDRg?si=ueCCcdj2Kema8zWG>

I Accidentally Pushed My API Key to GitHub — and Found Out Why That’s a Big Deal

 Alex Frankowski [Follow](#) 3 min read · Jul 18, 2025

 4     

This happened few weeks ago and I even shared it on LinkedIn.

I was working on a small project, trying to get an API integrated and working smoothly. Everything was going well — until I made one tiny mistake.

I pushed my API key to GitHub.

At first, I didn’t think much of it. I realized it a little later, deleted the key from my code, and assumed it was fine.

But later, while scrolling Instagram, I randomly came across a short video about a site called [UnsecuredAPIKeys.com](#) — and it completely changed how I think about these things.

<https://medium.com/@alexfrankowski54/i-accidentally-pushed-my-api-key-to-github-and-found-out-why-thats-a-big-deal-d53b5131d78e>

Unsecured API Keys REBORN

[+ Contribute a GitHub Token](#)

Help us find more exposed keys • Get **10x rate limits** in return!

🔥 Popular: [OpenAI](#) [GoogleAI](#) [SendGrid](#) [Random](#)

Random (Surprise me! 🎲)

 **Exposed Key Alert!**  [GoogleAI](#)

👁 40 eyes on this key

Status: Still Working! 🌸

Last Verified: 59 minutes ago ✅

First Exposed: 11/26/2025, 3:56:17 PM

Surviving in the wild: 1 month 🏃

Crime Scene: [Hackotberfest2025](#)

The Culprit: [@Open-Source-you](#) 🤖

Evidence Location:
[WeatherApp_Python/.env](#)

React to this find:

[😱 Shocked](#) [😬 Yikes](#) [😬 Facepalm](#) [👏 Impressive](#)

[Be a Hero 🦁](#) [Visit the Crime Scene 🔍](#) [Get Another Key 🎲](#)

📢 Share this find:

[✂ Share on X](#) [📄 Copy Share Text](#) [🔗 Share...](#)

 **140,196**
exposures

 **16**
online

<https://unsecuredapikeys.com>

Hall of Shame

The most exposed providers right now

GoogleAI

561

GoogleAI

561

TogetherAI

206

TogetherAI

206

De

Making bad developers famous since 2024™

105,862

Public Secrets

984

Still Exploitable

252

Found Today

2 hours ago

Latest Oopsie

* Refresh the page to update statistics

https://unsecuredapikeys.com

Unsecured API Keys

Live Verification Activity

Watch in real-time as our verification bot works through the queue. See which API providers are getting hammered and track hourly success rates!

Live Verification Feed

Filter: All Providers (50)

✓ DeepSeek (**c65c) verified valid **c65c

17:20:58

✓ DeepSeek (**3a37) verified valid **3a37

17:20:58

✓ StabilityAI (**nJDW) verified valid **nJDW

17:20:58

✓ StabilityAI (**TSRM) verified valid **TSRM

17:20:58

✓ DeepSeek (**8de1) verified valid **8de1

17:20:58

✓ DeepSeek (**0d66) verified valid **0d66

17:20:58

✓ StabilityAI (**92SL) verified valid **92SL

17:20:58

Showing last 50 activities

Hourly Verification Stats

17:20

TOTAL VERIFIED

898

SUCCESS RATE

98.1%

Valid

881

Invalid

0

Errors

0

GoogleAI

TOTAL

531

VALID

532

INVALID

0

ERRORS

0

100.2%

TogetherAI

TOTAL

170

VALID

170

INVALID

0

ERRORS

0

100%

DeepSeek

TOTAL

126

VALID

126

INVALID

0

ERRORS

0

100%

Stats reset at the top of each hour

140,196

exposures

17

online

9



r/googleo
Sandrrikk

4mo ago

...

Student hit with a \$55,444.78 Google Cloud bill after Gemini API key leaked on GitHub

Billing

Hi everyone, I never thought I'd end up in this kind of situation, but here I am. I signed up for Google Cloud with my student email and was only using the \$300 free credit they give you. Out of that, I had spent about \$80. That's it. I had more than \$220 left and I wasn't running anything serious, just doing small experiments for learning. On June 6, I accidentally pushed my API key to GitHub and I believed the repository was private (it was only visible in one commit, which I unfortunately didn't notice). At the time I didn't realize it, and since it was summer break, I wasn't even checking my student email. Then, on September 7, another GitHub user sent me a notification that my key had been public for a long time and others were abusing it. By that time, the damage was already done. When I checked my account, there was a \$55,444 in total. After that, I immediately revoked the Gemini API key. This is a sum that I never spent, never confirmed, and, to be honest, I never even imagined it was possible. In total, I received only two invoices: the first was for \$732 in June, however, the amount was not charged because my card had an expiration date of July 2025. If I had received a notification on my phone about a failed transaction, I would have immediately realized that something was wrong. But I didn't receive any such notification. The second invoice was for \$31,000+ in August, and then an additional \$21,000 was charged from September 1st to 7th. As soon as I discovered this, I immediately contacted Google Cloud Billing Support, filed a police report, and provided them with everything I could: usage logs, the GitHub links, screenshots all documents even when I revoked API key, attackers sent 14200+ , with 100 % rate failed requests in just 2 days. I also explained that my card on file had already expired, so the money could not be directly charged. Google reviewed my case, but the final answer was that the charges remain in effect. They were polite and empathetic, but the decision was final. No cancellation, no changes. Now I am receiving notices that if I don't pay within 10 days, the debt will be transferred to a collections agency, with possible additional fees. Looking at the situation from another perspective:

- I never confirmed these charges.
- I was only using the free \$300 credit.
- I was not checking my student email during summer break, so I did not know what was happening.
- My card had expired, so no money was ever charged. -I am a student from Georgia, where the average daily income is around \$15.
- There is no way I can pay \$55,000. This is much more than I will be able to earn in several decades.

I've seen posts online where Google forgave similar debts, sometimes fully, sometimes partially. This gives me a little hope, but in my case, I was not even given a symbolic relief. So I am asking: has anyone here ever dealt with such a large Google Cloud debt? Is there any way to escalate beyond the billing support team if the escalation manager told me that the decision is final? I am not trying to run away from responsibility, but I also don't want my life to be ruined because of something I didn't do myself. If anyone has advice, connections, or similar experience, I would be very grateful if you could write to me. And to any person starting to work with cloud services, please learn from my mistakes: protect your API keys, set spending limits, and check twice what you upload to GitHub. One small mistake can turn your life into a nightmare.

UPDATE 25 September -

I want to share some great news with you all. Following communication with the Google Cloud Billing Specialists, my case was reviewed again and the **total outstanding balance has been completely waived !**

I want to express my deepest gratitude to everyone for your sympathy and shared advice. Your support was very important to me.

I would also like to thank the **Google Billing Specialist team** for their service.

Thank you all again!

943

278

 1

 Share

https://www.reddit.com/r/googlecloud/comments/1noctxi/student_hit_with_a_5544478_google_cloud_bill/

▲ Lost \$300 due to an API key leak from "vibe coding" – Learn from my mistake

8 points by liulanggoukk 3 months ago | hide | past | favorite | 14 comments

I just learned an expensive lesson and wanted to share it here so others don't make the same mistake.

I recently lost \$300 because of an API key leak. It started with a surprise \$200 charge from Google Cloud, and when I looked into it, I found another \$100 charge from the day before. Both were for Gemini API usage that I never intentionally set up.

After digging, I discovered the issue: I had hard-coded an API key in a script that was part of a feature I ended up deprecating. The file was only in the codebase for two days, but that was enough for the key to leak. Google actually sent me alerts about unusual activity, but I missed them because they went to a less-frequently-checked email account.

Here's what I learned:

Never hardcode API keys - Use environment variables or a .env file, even for temporary code.

Set up billing alerts - Google Cloud (and other providers) let you set up alerts for unexpected charges.

Check all linked emails - Don't ignore notifications, even if they're sent to secondary accounts.

Don't rely solely on GitHub's secret scanning - It's useful, but renaming variables can bypass it.

This happened while I was experimenting with "vibe coding" (letting AI generate code quickly), but I realized too late that human oversight is still crucial, especially for security.

Hope this helps someone avoid the same costly mistake!

TL;DR: Hard-coded an API key in a deprecated script, key leaked, and I got charged \$300. Always use environment variables and set up billing alerts!

▲ scarface_74 3 months ago | next [-]

You learned the wrong lesson.

You should never specify API keys anywhere in your code or env files for GCP or AWS.

<https://cloud.google.com/docs/authentication/application-def...>

You still risk checking in your env file.

Doing it the correct way, your config is in your home directory locally far away from your repo and it finds the configuration automatically when running on GCP.

Even better when developing locally is assign environment variables to temporary access keys.

I'm being handwavy because I'm not a GCP guy. But on AWS, you do something similar by using "aws config" locally and using the IAM role attached to the VM, Lambda, etc so you never need to deploy with access keys.

This isn't meant to be an "AWS does it better comment". It looks like from my brief research, something similar is also best practice with GCP.

Issue 235: 25m loss at Kronos due to API key loss and three other API vulnerabilities

December 14, 2023

Share this article: [f](#) [t](#) [in](#)

This week, we have news of four API-related security vulnerabilities, including Kronos's \$25m loss. Other vulnerabilities include a malware threat of DDoS on Docker APIs, a report on vulnerabilities on WordPress and Netflix, and an API vulnerability found in the Ray AI framework. We also have an article on why APIs are fertile ground for attackers and on protecting APIs for online retail.

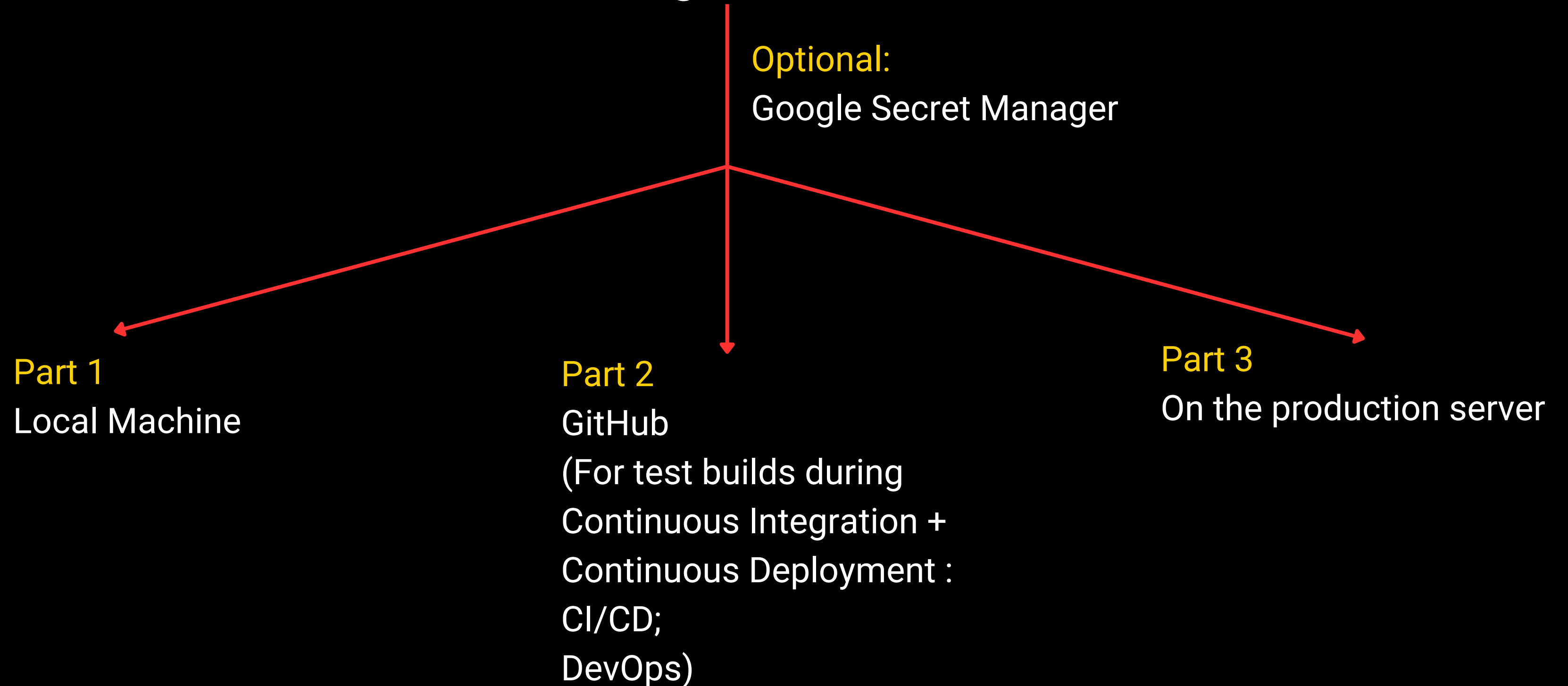
Breach: Kronos suffers \$25m loss involving API key loss

The [most important news this week](#) is the breach at the cryptocurrency fintech firm Kronos Research, who revealed in [a post on X](#) that they had suffered a loss estimated at \$25 million. The company admitted that the breach originated with "unauthorized access to some of their API keys," although it did not provide any further detail. Independent observers commented that they had seen Ethereum outflows from a wallet to a value of approximately \$25m, which seems to correlate with the disclosure.

This is yet another in a steady sequence of breaches involving the loss or leakage of API keys, particularly on cryptocurrency platforms. As ever, the advice is to protect the keys using vault or secure storage mechanisms and ensure they are invalidated in the event of loss or theft.

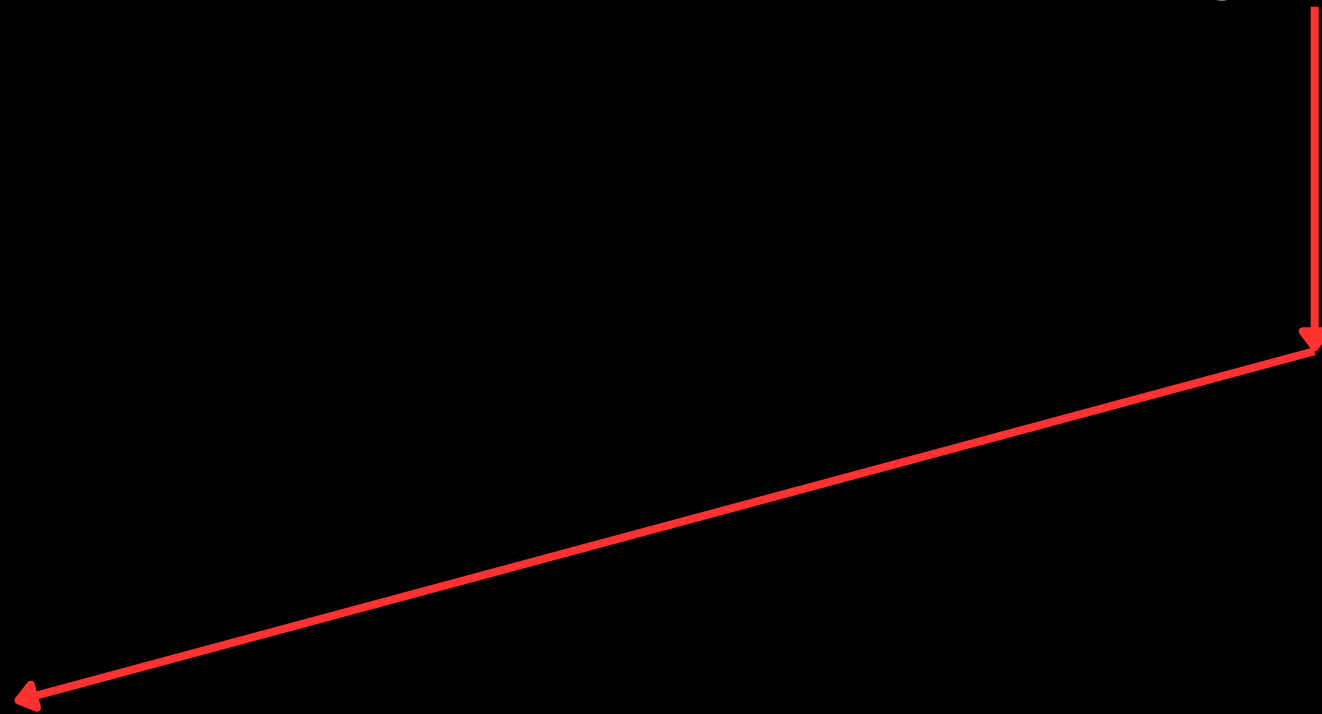
Solution: Three Pillars of Security

Storing the API



Solution: Three Pillars of Security

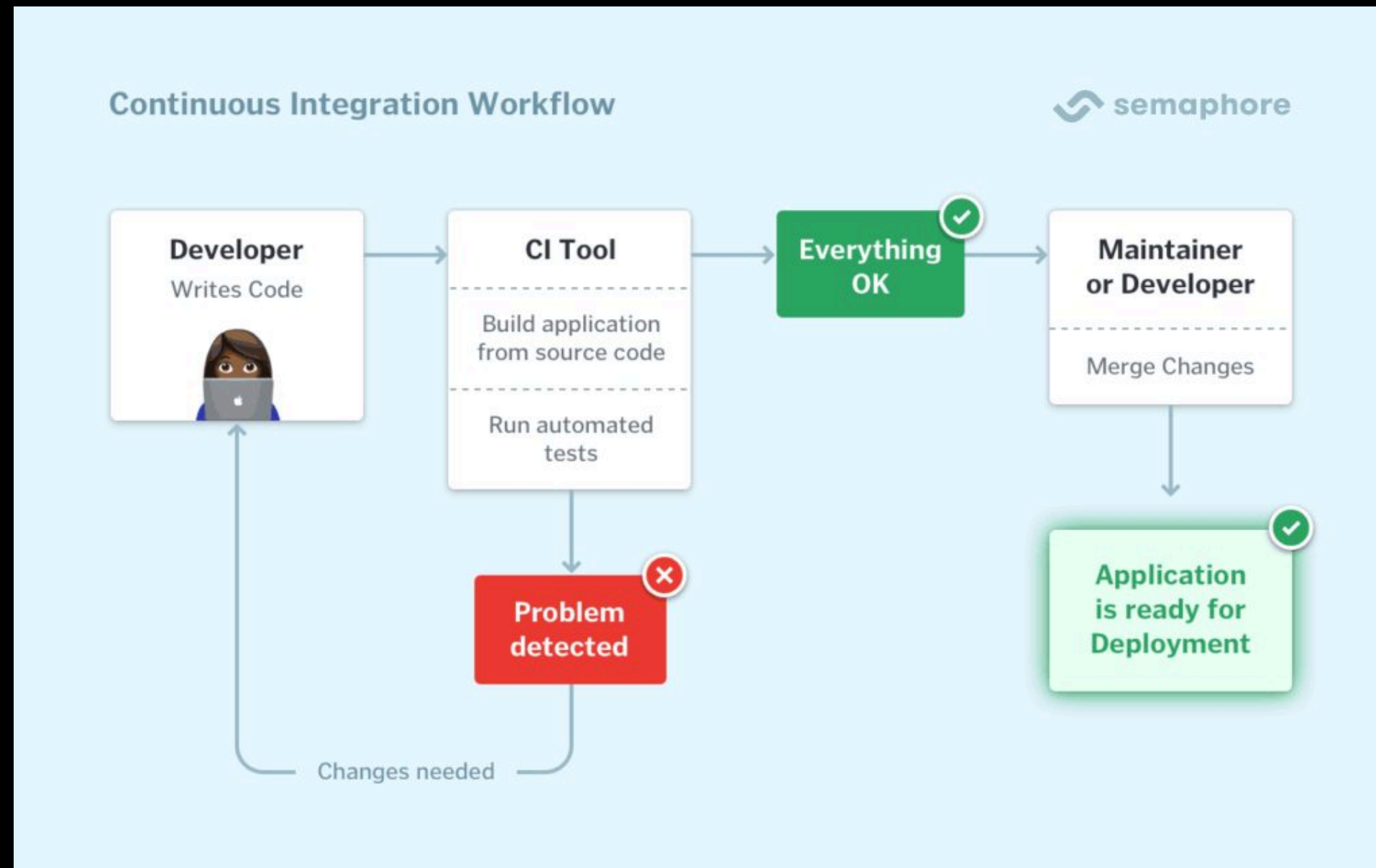
Storing the API



Part 1

Local Machine

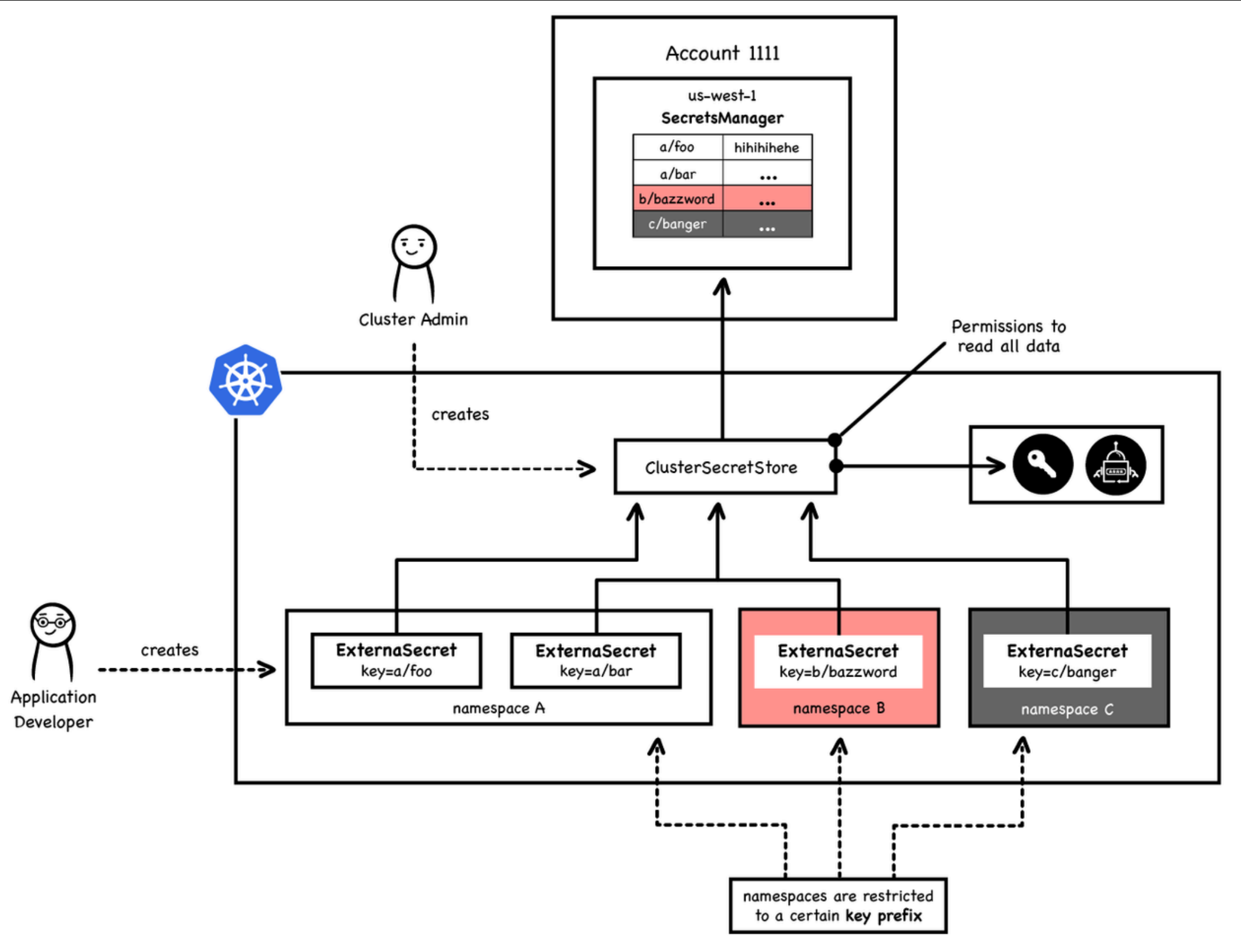
Focus of this
week's class.



<https://semaphore.io/continuous-integration>

Part 2
GitHub
(For test builds during
Continuous Integration +
Continuous Deployment :
CI/CD;
DevOps)

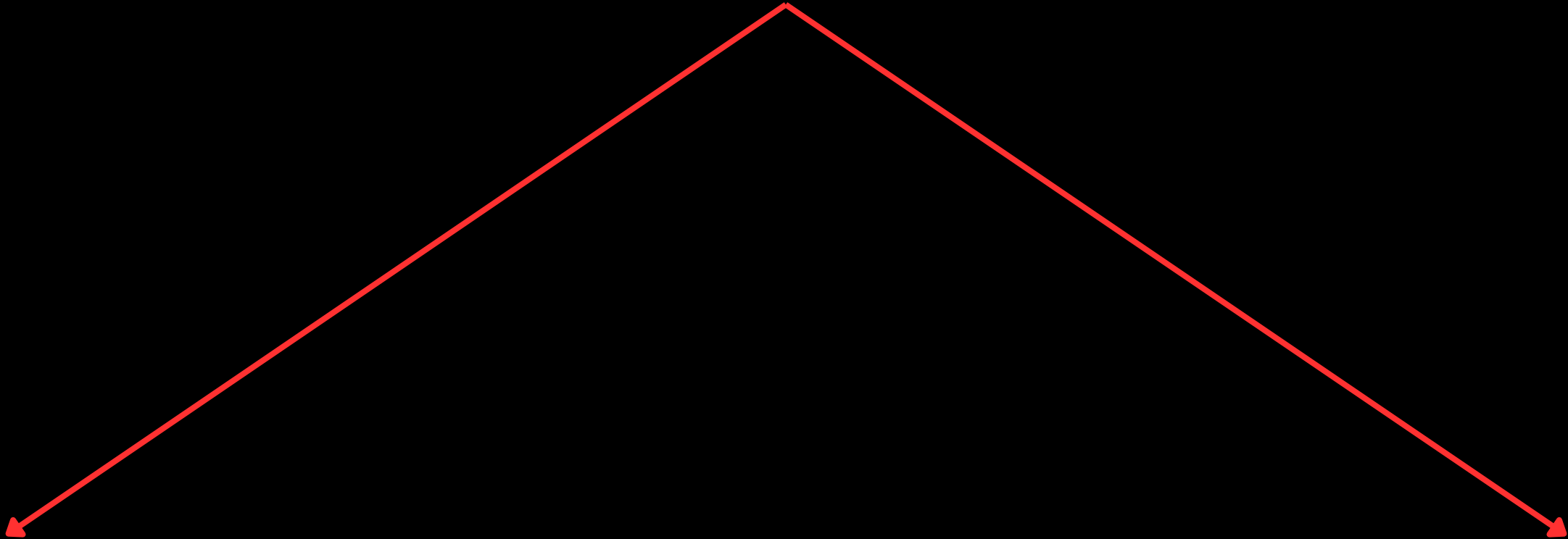
Optional:
Google Secret Manager



1. Directory Structure

- a. Deployment + Production Settings
- b. Database Refactoring

Environments



```
graph TD; A[Environments] --> B[Development]; A --> C[Production];
```

Development

Production

We will define both of them in a new settings folder.

This is called “split-setting” pattern.

```
settings.py x
26
27 # SECURITY WARNING: don't run with debug turned on in production!
28 DEBUG = True
29
30 ALLOWED_HOSTS = []
31
```

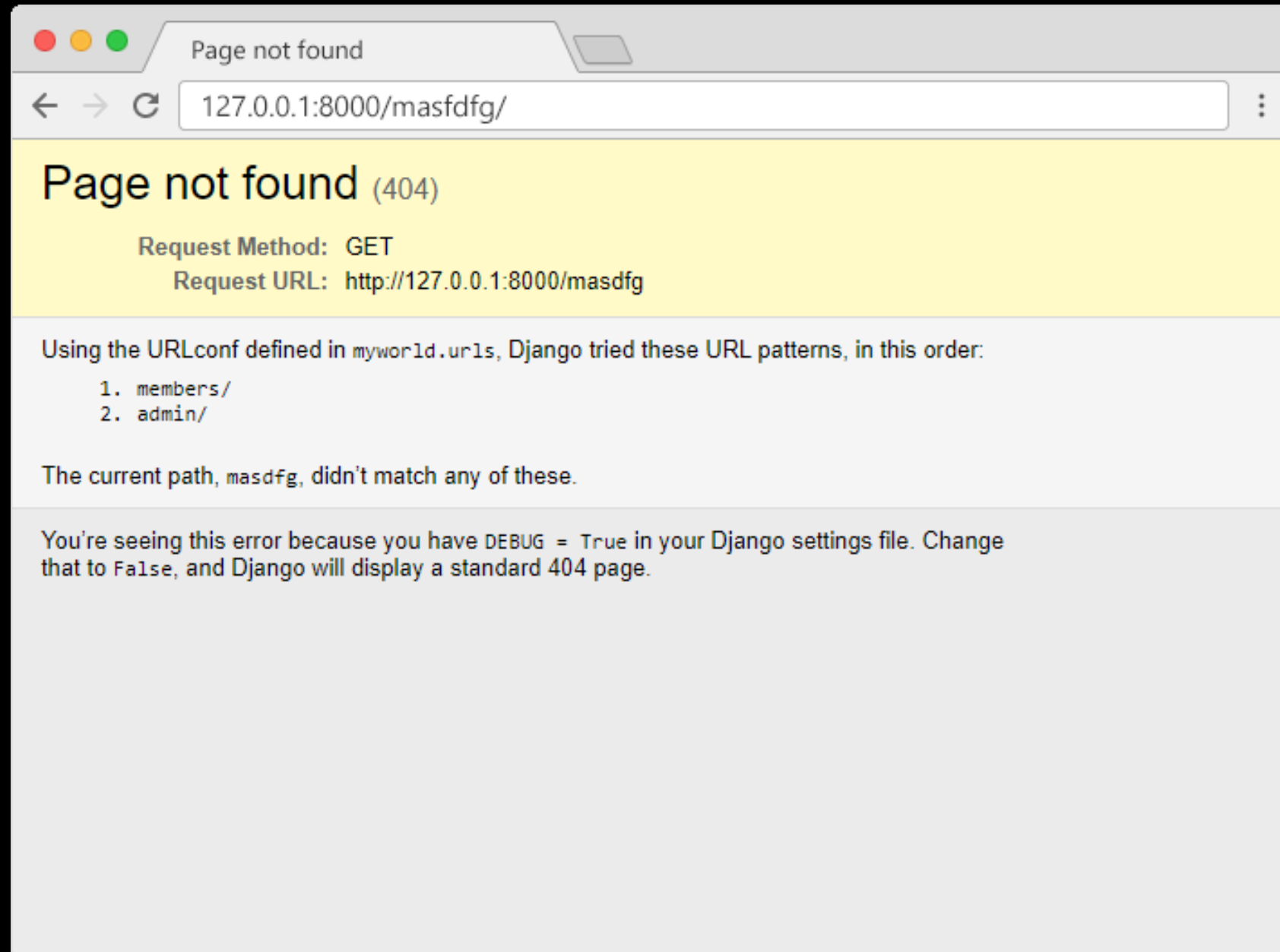
Debug = True (Used for development server)

Debug = False (Used for production server)

Debug = True shows current application settings,
file paths if any errors occur.

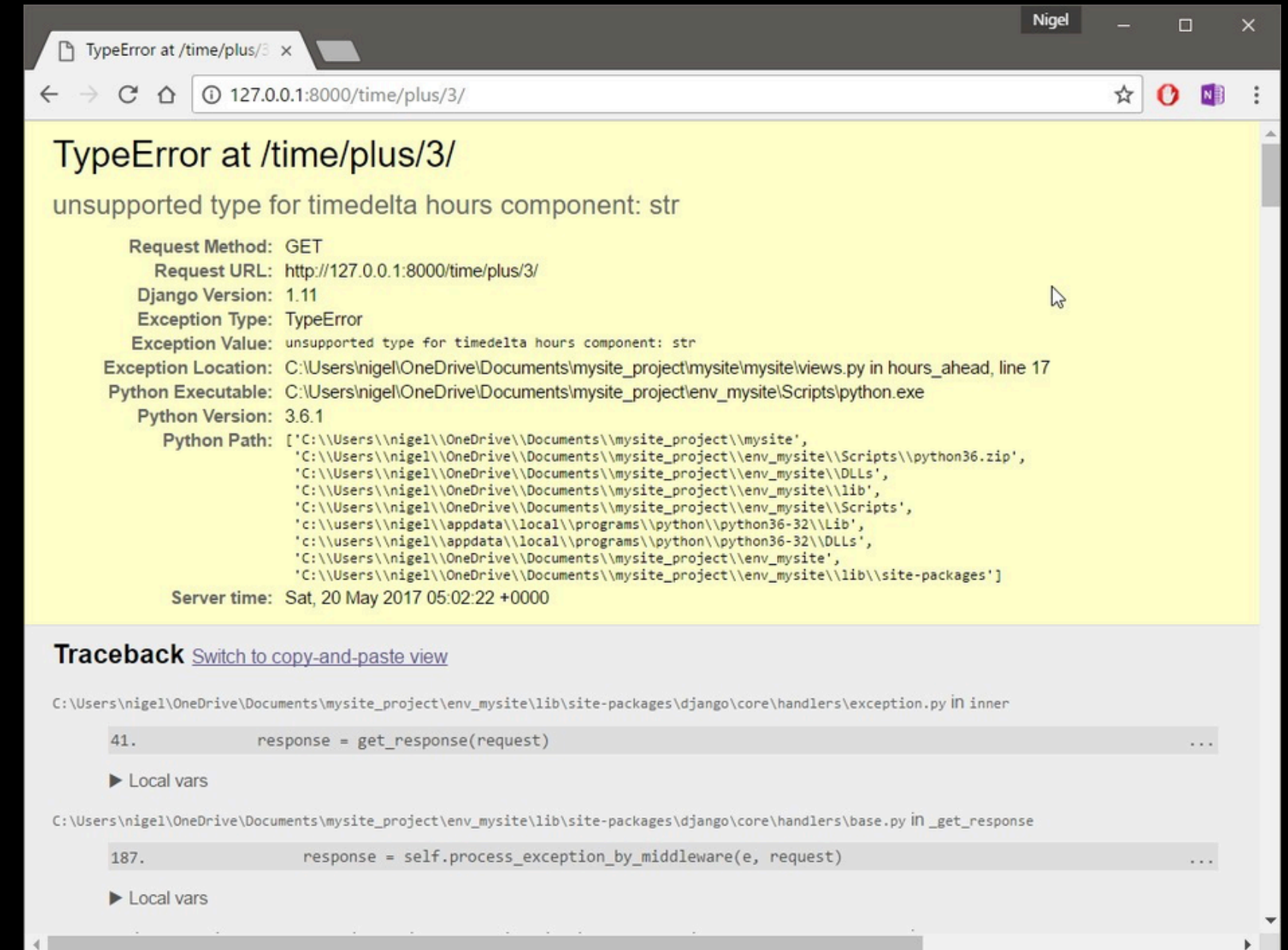
If used in production, the world will see the
diagnosis of error which we don't want.

No Bug



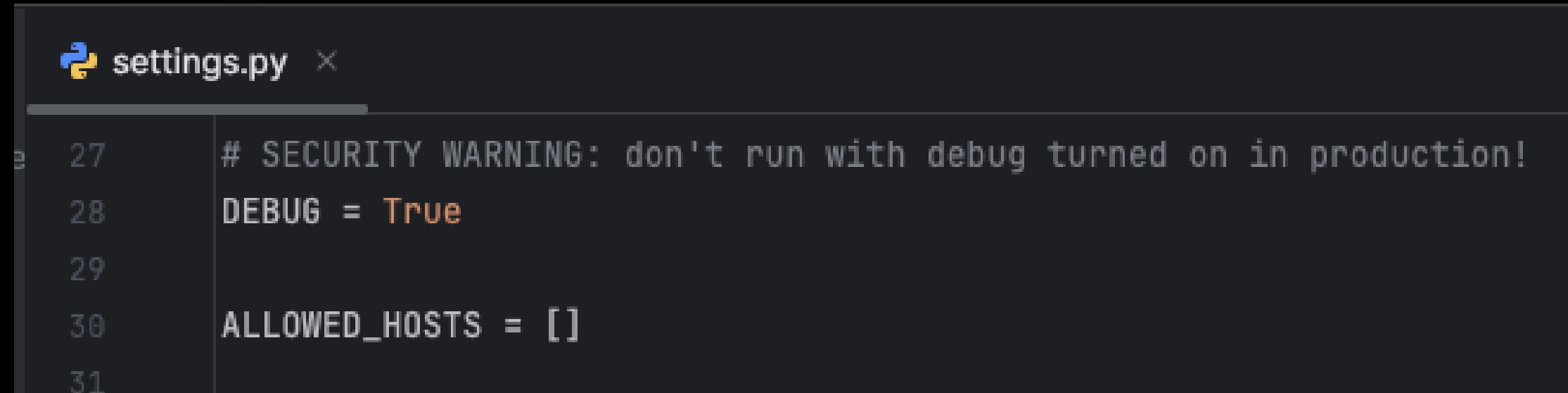
https://www.w3schools.com/django/django_404.php

Bug



<https://www.vskills.in/certification/tutorial/djangos-pretty-error-pages/>

In case `Debug = False`, we'll also need to define the url where our website will be hosted.

A screenshot of a code editor window titled 'settings.py' with a Python icon and a close button. The editor shows a dark-themed interface with a line number column on the left. The code is as follows:

```
27 # SECURITY WARNING: don't run with debug turned on in production!
28 DEBUG = True
29
30 ALLOWED_HOSTS = []
31
```

Now, we might be using the same or another database.

And this is where we'll define them!

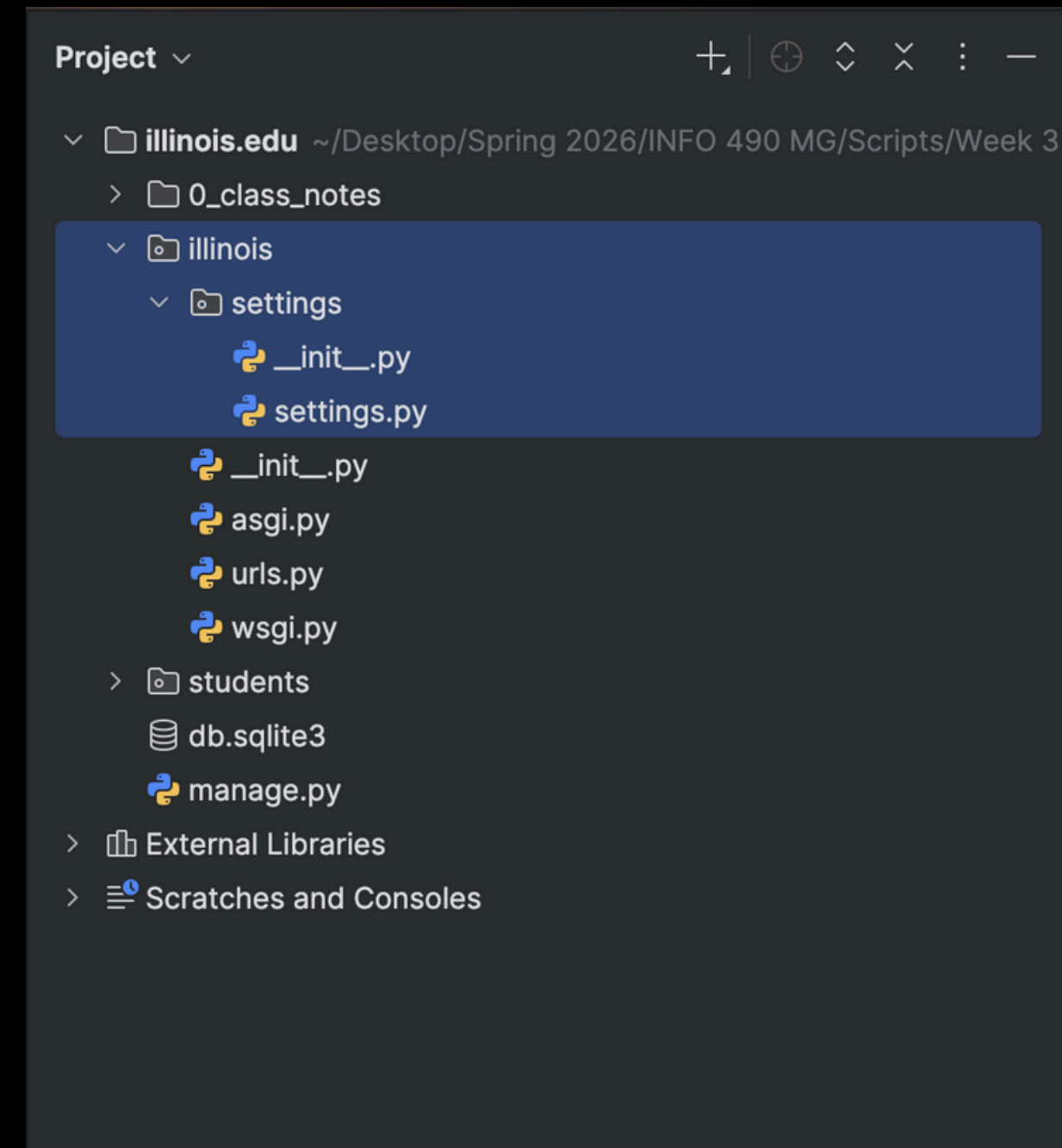
```
settings.py ×  
81 # Database  
82 # https://docs.djangoproject.com/en/5.2/ref/settings/#databases  
83  
84 DATABASES = {  
85     'default': {  
86         'ENGINE': 'django.db.backends.sqlite3',  
87         'NAME': BASE_DIR / 'db.sqlite3',  
88     }  
89 }  
90
```


1.

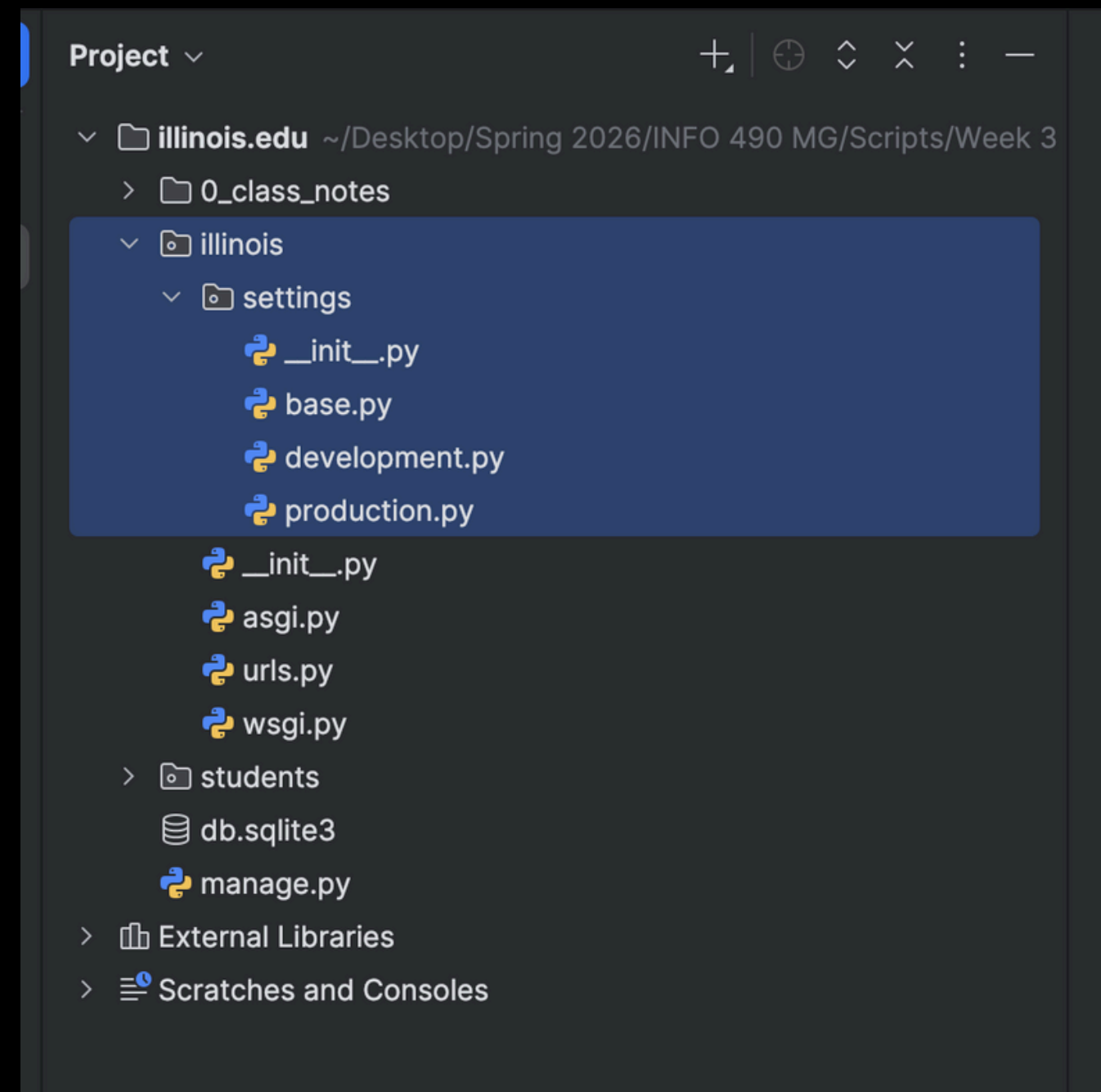
Let's make it organized.

We'll create a settings folder and
put development and production
settings in it!

- Create a folder settings, and in it `create a __init__.py` file.
- This will indicate to python that we're creating a module.
- Now shift the main `settings.py` file into this new folder.



- Now rename the settings.py to `base.py`
- And create two new files:
 - `development.py`
 - `production.py`



```
illinois.edu [~/Mohit Gupta/Acade

base.py × development.py production.py wsgi.py asgi.py
13
14 # SECURITY WARNING: don't run with debug turned on in production!
15 # Cut and Paste these in development.py and production.py
16 DEBUG = True
17 ALLOWED_HOSTS = []
18
19
```

Cut and paste the variables in these two images into the development.py and production.py

```
base.py × development.py production.py wsgi.py asgi.py
3 60
61
62 # Database
63 # https://docs.djangoproject.com/en/5.2/ref/settings/#databases
64 # Cut and paste in development.py and production.py
65 DATABASES = {
66     'default': {
67         'ENGINE': 'django.db.backends.sqlite3',
68         'NAME': BASE_DIR / 'db.sqlite3',
69     }
70 }
```

```
base.py development.py × production.py
1 # Paste in: illinois/settings/development.py
2
3 # We will first import everything from base.py (our new
4 from .base import *
5
6 # SECURITY WARNING: don't run with debug turned on in pr
7 DEBUG = True
8 ALLOWED_HOSTS = ['localhost', '127.0.0.1']
9
10 # Database for the development server
11 DATABASES = {
12     'default': {
13         'ENGINE': 'django.db.backends.sqlite3',
14         # put the DB in project-root/data/db.sqlite3
15         'NAME': BASE_DIR / 'db.sqlite3',
16     }
17 }
```

```
base.py development.py production.py ×
1 # Paste in: illinois/settings/production.py
2
3 # We will first import everything from base.py (our new
4 from .base import *
5
6 DEBUG = False
7
8 # Replace it with your name:
9 ALLOWED_HOSTS = ['localhost', '127.0.0.1']
10
11 # Database for the development server
12 DATABASES = {
13     'default': {
14         'ENGINE': 'django.db.backends.sqlite3',
15         # put the DB in project-root/data/db.sqlite3
16         'NAME': BASE_DIR / 'db.sqlite3',
17     }
18 }
```

Our two files should now look like this.

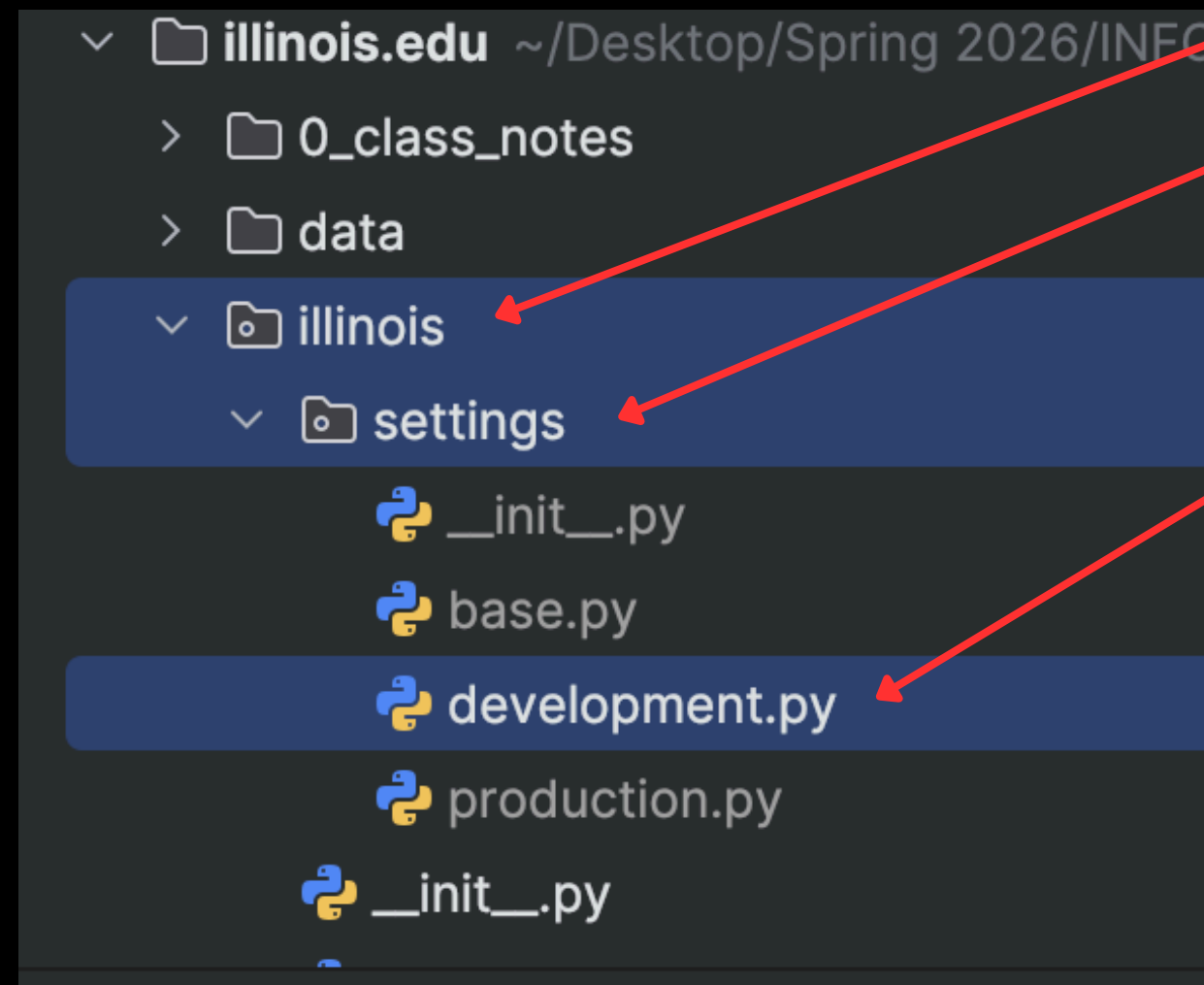
```
Terminal Local x + v
(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu % python manage.py runserver
CommandError: You must set settings.ALLOWED_HOSTS if DEBUG is False.
(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu %
```

You'll get now an error when you do:
`python manage.py runserver`

Any guesses why?


```
Terminal Local x + v
(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu % python manage.py runserver
CommandError: You must set settings.ALLOWED_HOSTS if DEBUG is False.

(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu % python manage.py runserver --settings=illinois.settings.development
Watching for file changes with StatReloader
Performing system checks...
```



Given that we now changed the location of the settings, we'll need to specify the location of settings in the command.

Activity:
Now try running the production server.

Your server should be now up and running.

Terminal Local × + ▾

```
(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu % python manage.py runserver
```

```
CommandError: You must set settings.ALLOWED_HOSTS if DEBUG is False.
```

```
(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu % python manage.py runserver --settings=illinois.settings.development
```

```
Watching for file changes with StatReloader
```

```
Performing system checks...
```

```
System check identified no issues (0 silenced).
```

```
January 09, 2026 - 01:21:28
```

```
Django version 5.2.8, using settings 'illinois.settings.development'
```

```
Starting development server at http://127.0.0.1:8000/
```

```
Quit the server with CONTROL-C.
```

```
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
```

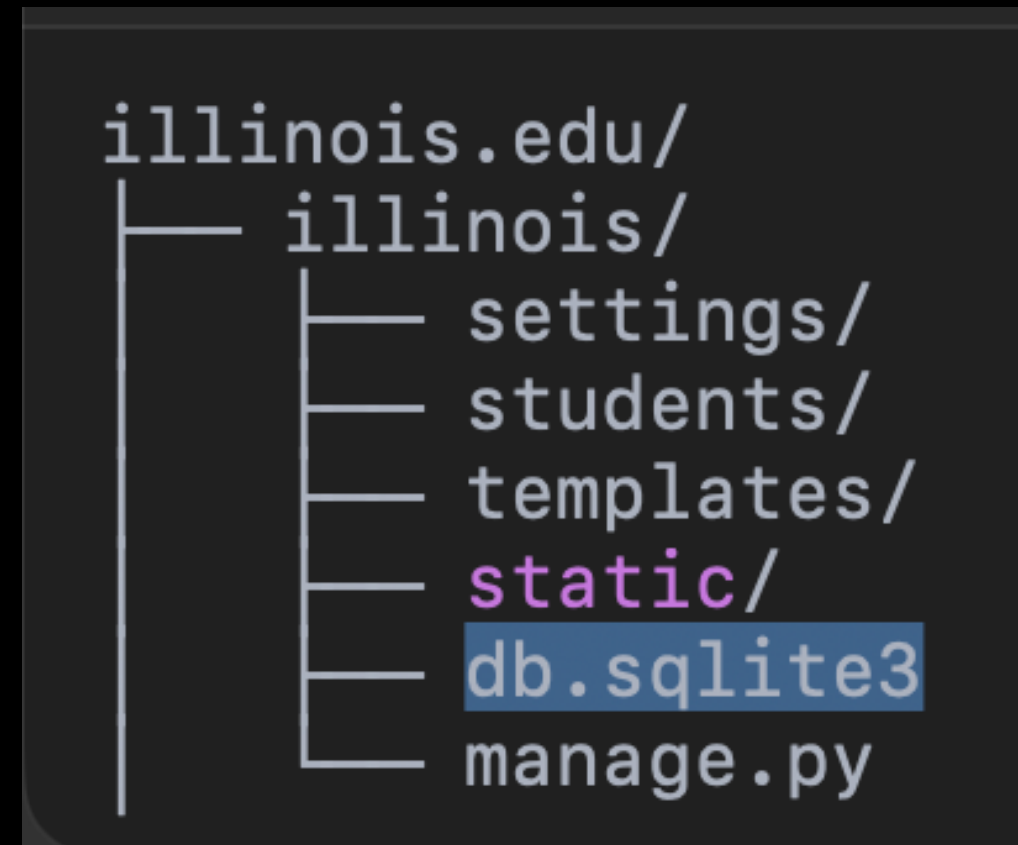
```
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
```

```
█
```

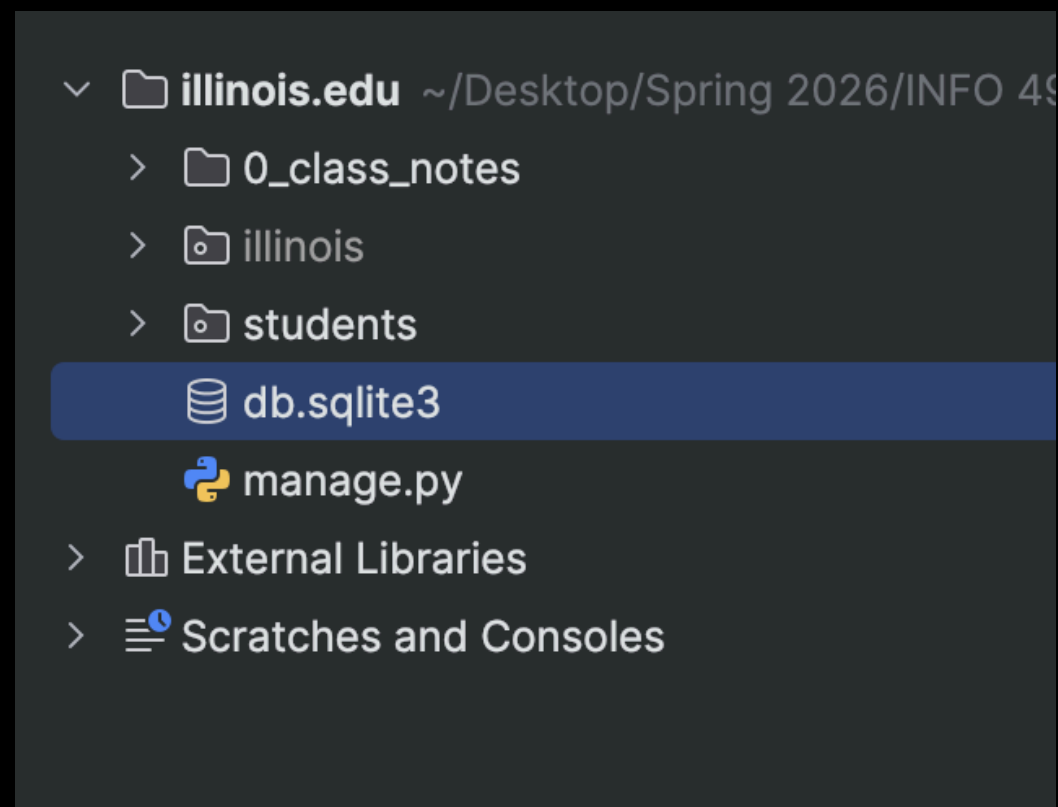

2.

Let's change the location of `db.sqlite3`

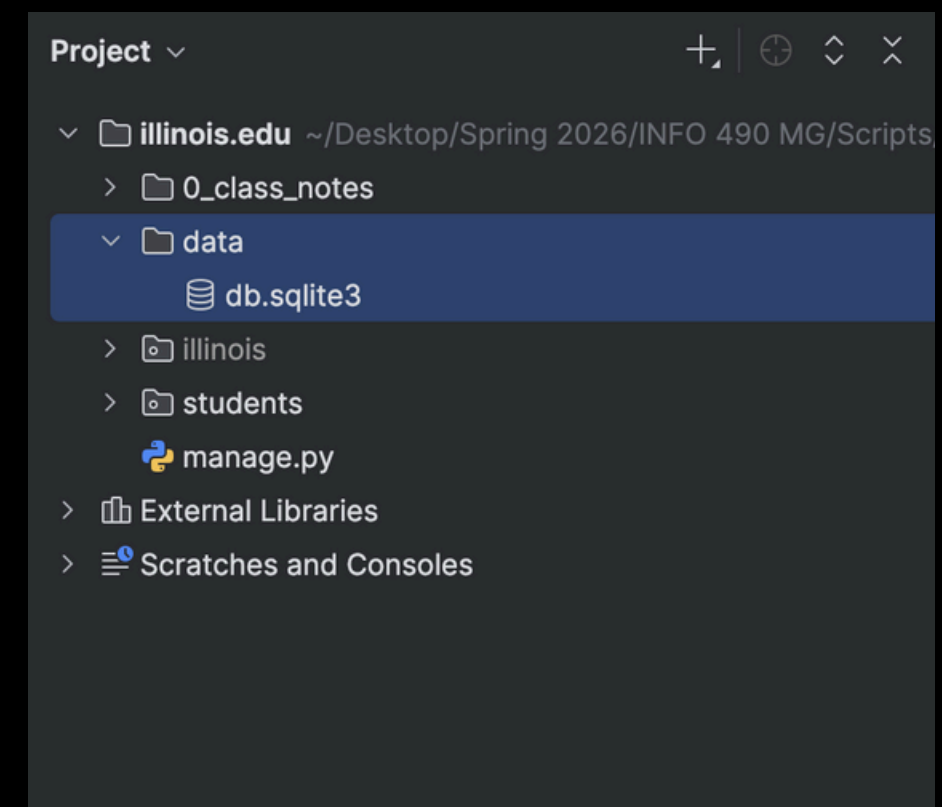
- Now, let's also make this more organized and shift the data into a new folder called 'data'
- This was the old hierarchy structure. So the new one looks like (next slide)



Old directory structure



Old directory structure



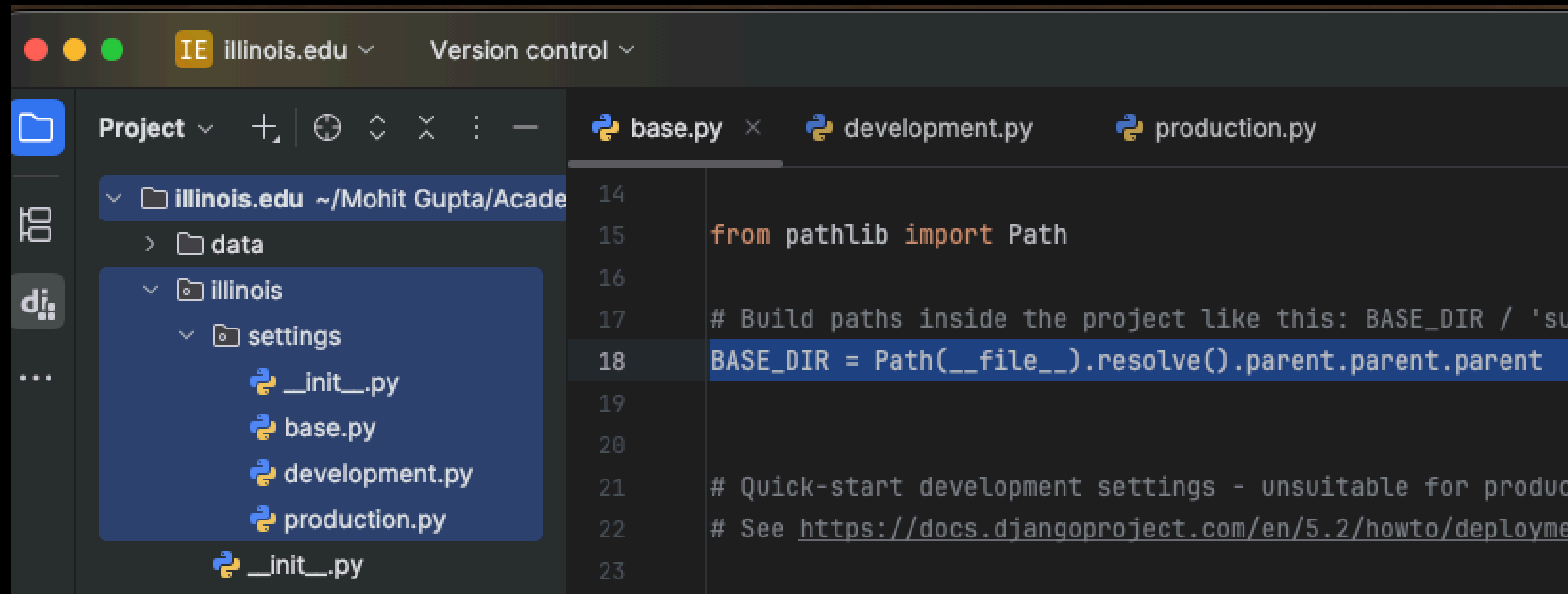
New directory structure

Because we changed the location
of settings.py,
the locations mentioned inside it
also needs to be changed.

This is because...

...our `BASE_DIR` in `base.py` follows **relative path structure** with reference to itself.

```
illinois.edu/           ← project root (where manage.py lives)
├── illinois/            ← Django app folder
│   ├── settings/
│   │   ├── base.py
│   │   ├── development.py
│   │   └── production.py
│   ├── templates/
│   └── ...
├── templates/           ← global templates folder
└── manage.py
```

- We are currently three levels deep inside our new location for settings.py (now base.py):
- `illinois.edu/illinois/settings/base.py`

- `BASE_DIR` correctly points to where `manage.py` lives.

```
Path(__file__).resolve() → /illinois.edu/illinois/settings/base.py
.parent                  → /illinois.edu/illinois/settings
.parent                  → /illinois.edu/illinois
```

- But given that `manage.py` lives one more level up, we need to add one more `.parent`

- Now, just by adding one more **.parent**, we have correctly adjusted the location of the **templates folder** and **manage.py**

```
base.py × development.py production.py
14
15 from pathlib import Path
16
17 # Build paths inside the project like this: BASE_DIR / 'subdir'.
18 BASE_DIR = Path(__file__).resolve().parent.parent.parent
19
```

↓

Path(__file__).resolve()	→ /illinois.edu/illinois/settings/base.py
.parent	→ /illinois.edu/illinois/settings
.parent	→ /illinois.edu/illinois
.parent	→ /illinois.edu ✓ (project root)

- Given that we've put our **db.sqlite3** into a new **data folder**, let's also mention that in the respective variable.
- So, the new location of the **db.sqlite3** becomes:
- **BASE_DIR/data/db.sqlite3**
 - or
- **illinois/data/db.sqlite3**

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'data' / 'db.sqlite3',  
    }  
}
```

Make little tweaks in the Database's path in the `development.py` and `production.py`

Development

```
base.py  development.py  production.py
1  # Paste in: illinois/settings/development.py
2
3  # We will first import everything from base.py (our new sett
4  from .base import *
5
6  # SECURITY WARNING: don't run with debug turned on in pro
7  DEBUG = True
8  ALLOWED_HOSTS = ['localhost', '127.0.0.1']
9
10 # Database for the development server
11 DATABASES = {
12     'default': {
13         'ENGINE': 'django.db.backends.sqlite3',
14         # put the DB in project-root/data/db.sqlite3
15         'NAME': BASE_DIR / 'data' / 'db.sqlite3',
16     }
17 }
```

Production

```
base.py  development.py  production.py
1  # Paste in: illinois/settings/production.py
2
3  # We will first import everything from base.py (our new sett
4  from .base import *
5
6  DEBUG = False
7
8  # Enter your PythonAnywhere account here in the allowed host
9  # Replace it with your name:
10 ALLOWED_HOSTS = ['localhost', '127.0.0.1']
11
12 # Database for the development server
13 DATABASES = {
14     'default': {
15         'ENGINE': 'django.db.backends.sqlite3',
16         # put the DB in project-root/data/db.sqlite3
17         'NAME': BASE_DIR / 'data' / 'db.sqlite3',
18     }
19 }
```

Our two files should now look like this.

Now try running the development and production servers.

2. Environment Variables (Local Machine)

We use .env or similar approaches to not push our secret keys or the APIs to the GitHub. Otherwise we've seen the consequences in the intital slides of this lecture.

```
1 .env
... @@ -0,0 +1 @@
1 + API_KEY=a54df9cd-a513-40cb-bffc-07351079651c
```

*developer



```
1 .env
... @@ -0,0 +1 @@
1 + API_KEY=a54df9cd-a513-40cb-bffc-07351079651c
```

*manager



<https://youtube.com/shorts/zgR4GmrcDRg?si=ueCCcdj2Kema8zWG>

Deployment checklist ¶

The internet is a hostile environment. Before deploying your Django project, you should take some time to review your settings, with security, performance, and operations in mind.

Django includes many [security features](#). Some are built-in and always enabled. Others are optional because they aren't always appropriate, or because they're inconvenient for development. For example, forcing HTTPS may not be suitable for all websites, and it's impractical for local development.

Performance optimizations are another category of trade-offs with convenience. For instance, caching is useful in production, less so for local development. Error reporting needs are also widely different.

The following checklist includes settings that:

- must be set properly for Django to provide the expected level of security;
- are expected to be different in each environment;
- enable optional security features;
- enable performance optimizations;
- provide error reporting.

Many of these settings are sensitive and should be treated as confidential. If you're releasing the source code for your project, a common practice is to publish suitable settings for development, and to use a private settings module for production.

<https://docs.djangoproject.com/en/6.0/howto/deployment/checklist>

Run `manage.py check --deploy` ¶

Some of the checks described below can be automated using the `check --deploy` option. Be sure to run it against your production settings file as described in the option's documentation.

Switch away from `manage.py runserver` ¶

The `runserver` command is not designed for a production setting. Be sure to switch to a production-ready WSGI or ASGI server. For a few common options, see [WSGI servers](#) or [ASGI servers](#).

Critical settings ¶

`SECRET_KEY` ¶

The secret key must be a large random value and it must be kept secret.

Make sure that the key used in production isn't used anywhere else and avoid committing it to source control. This reduces the number of vectors from which an attacker may acquire the key.

Instead of hardcoding the secret key in your settings module, consider loading it from an environment variable:

```
import os

SECRET_KEY = os.environ["SECRET_KEY"]
```

or from a file:

```
with open("/etc/secret_key.txt") as f:
    SECRET_KEY = f.read().strip()
```

If rotating secret keys, you may use `SECRET_KEY_FALLBACKS`:

```
import os

SECRET_KEY = os.environ["CURRENT_SECRET_KEY"]
SECRET_KEY_FALLBACKS = [
    os.environ["OLD_SECRET_KEY"],
]
```


Two Solutions:

`.env`

- best & industry standard
- comes from Unix, Docker, Heroku, Kubernetes, and cloud platforms
- not native to Django or python
- It is universal way of saying to cloud: Here are my env variables for this app.
- Cloud tools can read it, load it, validate it, document it, sync it, rotate it

VS

`env.py`

- basic python file;
- file name can be variable
- cannot be used by Docker, Github Actions, Render, Kubernetes
- Invisible to infrastructure

env.py

- basic python file;
- name can be variable
- has cons
- stores keys in systems environment which can leak in other projects

```
import os

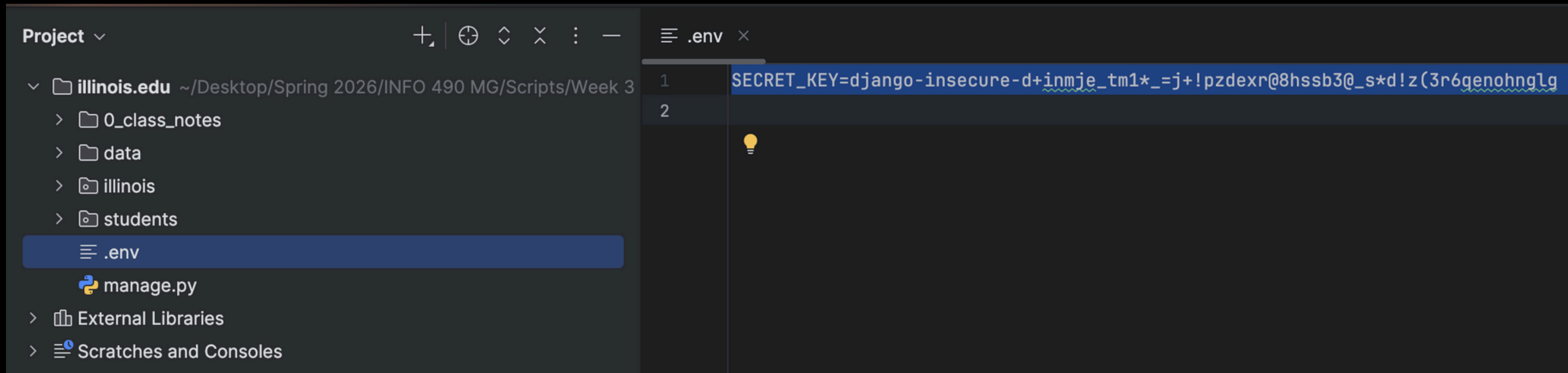
os.environ["DJANGO_SETTINGS_MODULE"] = "illinois.settings.development"
os.environ["DJANGO_SECRET_KEY"] = ""
os.environ["DATABASE_URL"] = ""
os.environ["OPENAI_API_KEY"] = ""
```

How it will be stored in the new python file.

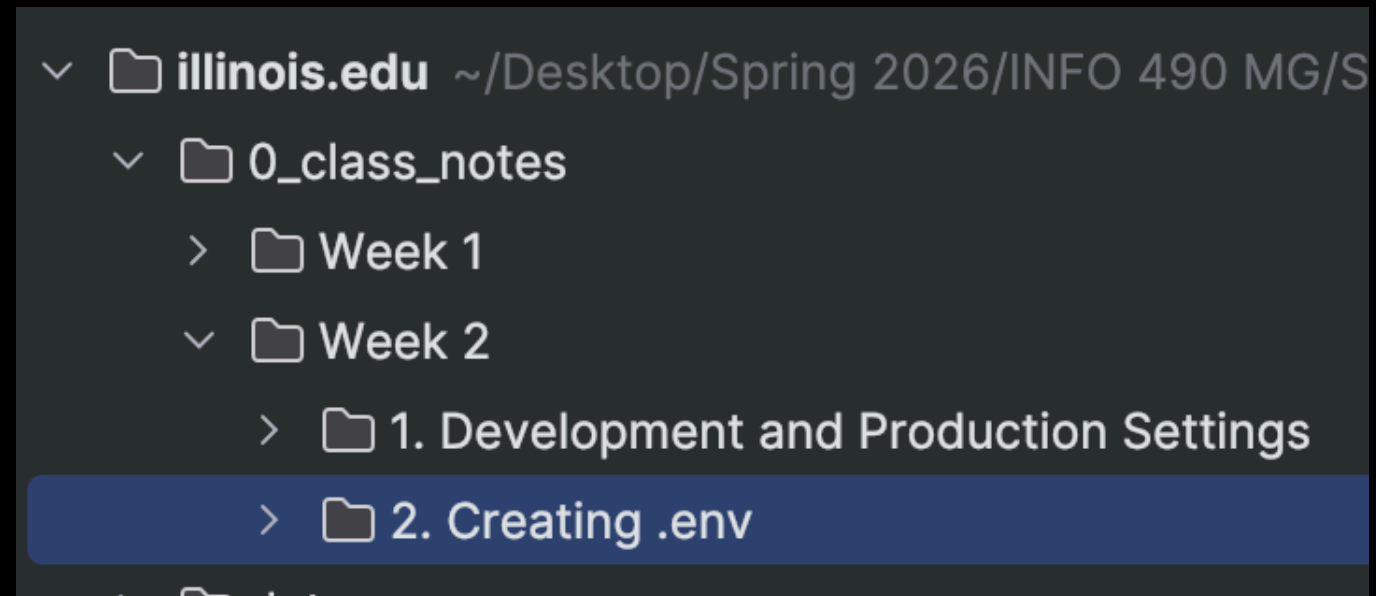
This won't be committed to GitHub

.env

- best & industry standard
- comes from Unix, Docker, Heroku, Kubernetes, and cloud platforms
- not native to Django or python
- It is universal way of saying to cloud: Here are my env variables for this app.
- Cloud tools can read it, load it, validate it, document it, sync it, rotate it



The screenshot shows a code editor interface. On the left, a project explorer shows a folder named 'illinois.edu' with subfolders '0_class_notes', 'data', 'illinois', and 'students'. Below these is a file named '.env', which is selected. To the right of the explorer, the '.env' file is open in the editor. It contains a single line of text: `SECRET_KEY=django-insecure-d+inmje_tm1*_=j+!pzdexr@8hssb3@_s*d!z(3r6qenohnglg`. The text is highlighted in blue. Below the editor, there is a lightbulb icon, indicating a suggestion or warning.



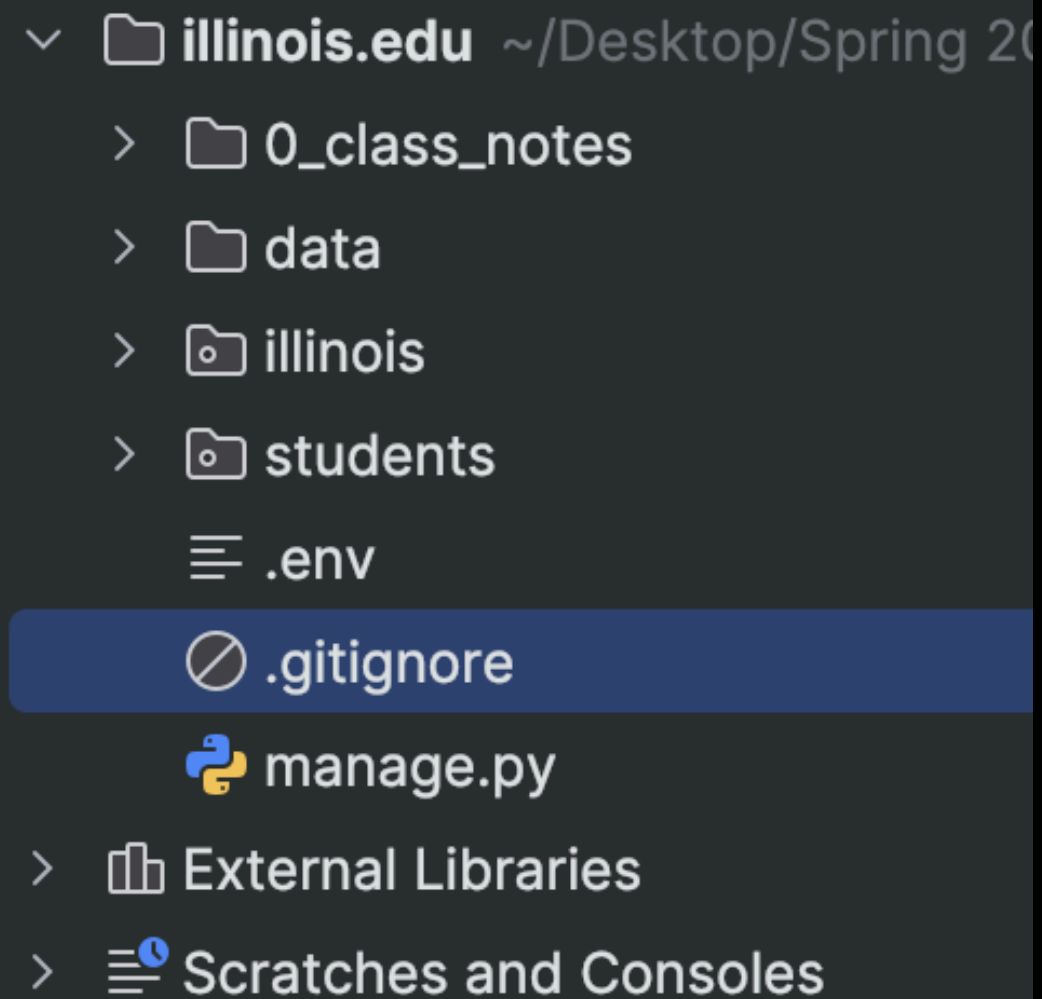
Let's open the scripts and notes for .env in the tutorial-scripts and begin coding

3. Github

- a. .gitignore
- b. setup

1. Create a .gitignore file in home directory

In it, put all the files we do not want to push to GitHub like **.env**



Find this file in notes:

.gitignore

and paste it here

2. Setup: Push changes to GitHub

On GitHub, you'll see that no .env files are uploaded as they were mentioned in **.gitignore**.

Create a repository on Github

Now create a new GitHub Repository before proceeding

Github.com

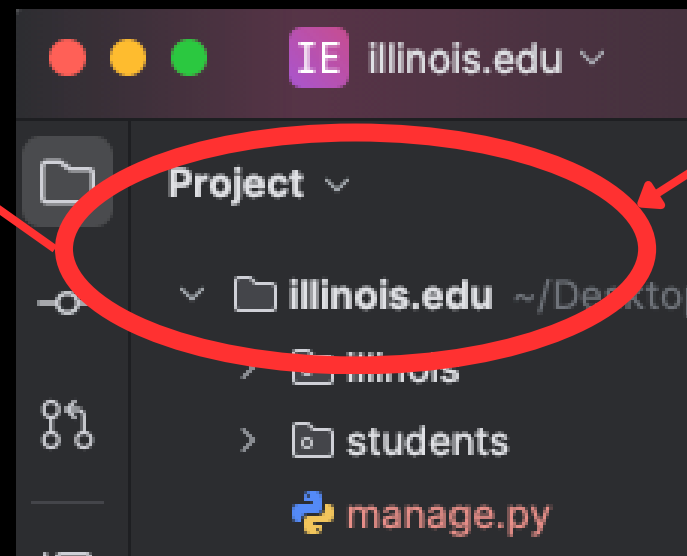
Create repository with the same name as current directory

For file structure on right, repository name (private or public):

Imp: Don't initialize with a README

File name same as your project name

illinois.edu

A screenshot of the GitHub 'Create a new repository' form. The form is divided into two sections: 'General' and 'Configuration'. In the 'General' section, the 'Repository name' field is highlighted with a red circle and contains the text 'illinois.edu'. Below this field, a green checkmark indicates 'illinois.edu is available.'. The 'Owner' field shows the user '27guptamohit'. The 'Description' field contains the text 'Github demonstration for INFO 490 MG'. In the 'Configuration' section, the 'Choose visibility' dropdown is set to 'Private', the 'Add README' toggle is 'Off', the 'Add .gitignore' dropdown is set to 'No .gitignore', and the 'Add license' dropdown is set to 'No license'. A green 'Create repository' button is at the bottom right. A red arrow points from the 'illinois.edu' text in the file explorer to the 'Repository name' field.

Now we can begin pushing the code on GitHub given that we're now using .env

The screenshot shows the GitHub interface for a repository named 'illinois.edu' (Private). At the top, there are buttons for 'Watch' (0), 'Fork' (0), and 'Star' (0). Below this, there are two main sections: 'Set up GitHub Copilot' and 'Add collaborators to this repository'. The 'Set up GitHub Copilot' section includes a button 'Get started with GitHub Copilot'. The 'Add collaborators to this repository' section includes a button 'Invite collaborators'. Below these, there is a 'Quick setup' section with a button 'Set up in Desktop' and a button 'HTTPS SSH' next to the repository URL 'https://github.com/27guptamohit/illinois.edu.git'. Below the 'Quick setup' section, there is a section titled '...or create a new repository on the command line' with a code block containing the following commands:

```
echo "# illinois.edu" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/27guptamohit/illinois.edu.git
git push -u origin main
```

 Below this, there is a section titled '...or push an existing repository from the command line' with a code block containing the following commands:

```
git remote add origin https://github.com/27guptamohit/illinois.edu.git
git branch -M main
git push -u origin main
```

 At the bottom, there is a 'ProTip!' section with a lightbulb icon and the text: 'Use the URL for this page when adding GitHub as a remote.'

illinois.edu Private

Watch 0 Fork 0 Star 0

Set up GitHub Copilot
Use GitHub's AI pair programmer to autocomplete suggestions as you code.
Get started with GitHub Copilot

Add collaborators to this repository
Search for people using their GitHub username or email address.
Invite collaborators

Quick setup — if you've done this kind of thing before
Set up in Desktop or HTTPS SSH `https://github.com/27guptamohit/illinois.edu.git`
Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

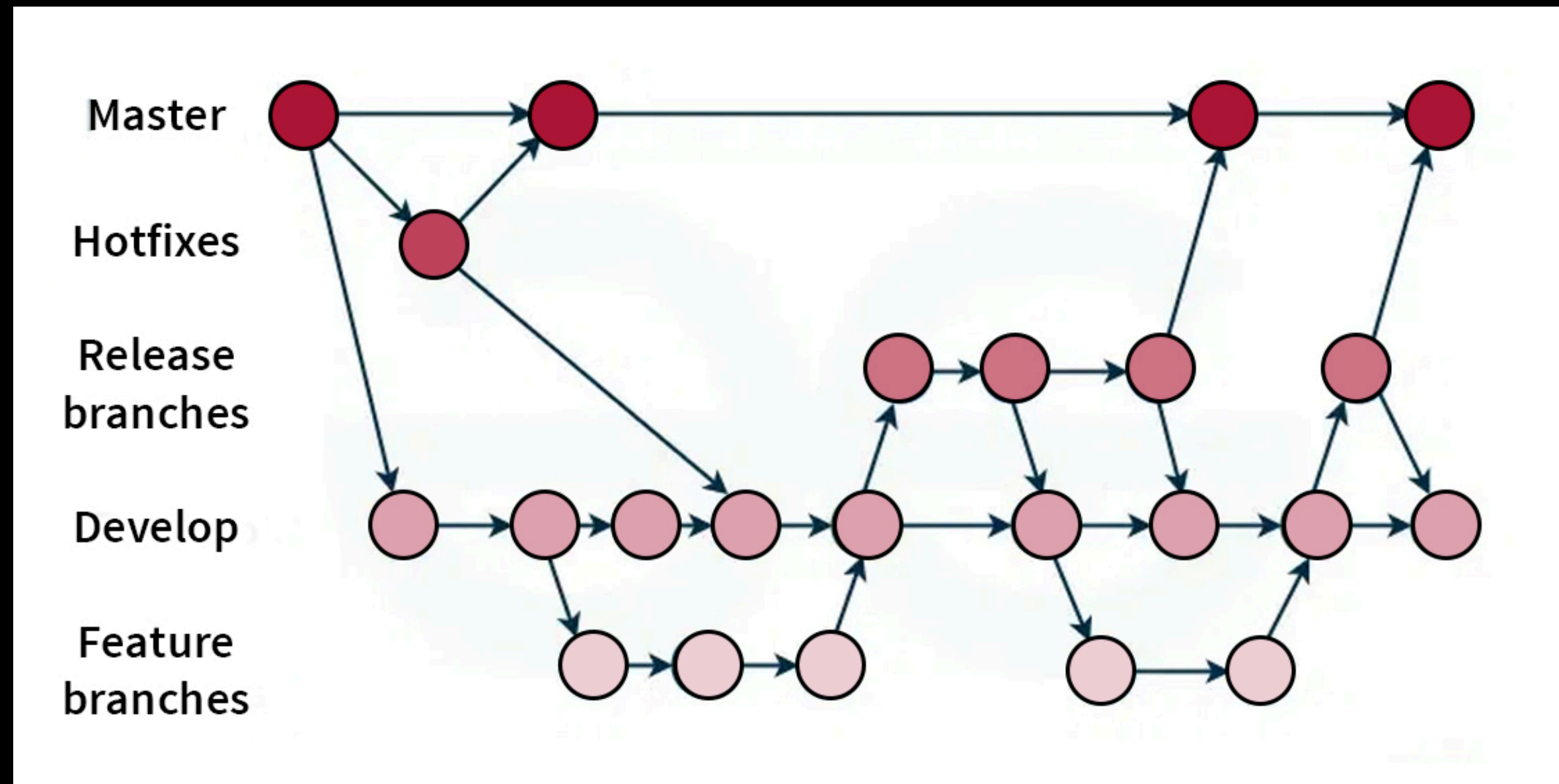
```
echo "# illinois.edu" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/27guptamohit/illinois.edu.git
git push -u origin main
```

...or push an existing repository from the command line

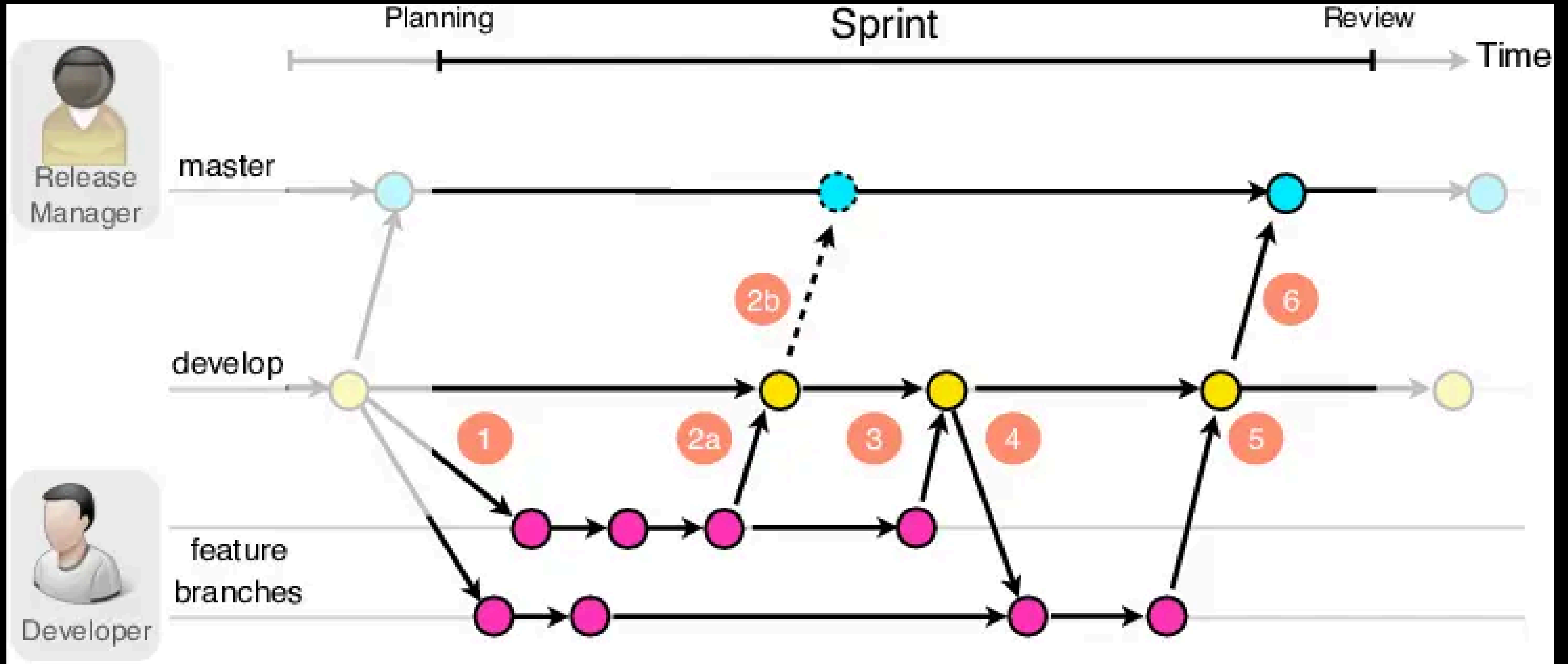
```
git remote add origin https://github.com/27guptamohit/illinois.edu.git
git branch -M main
git push -u origin main
```

ProTip! Use the URL for this page when adding GitHub as a remote.

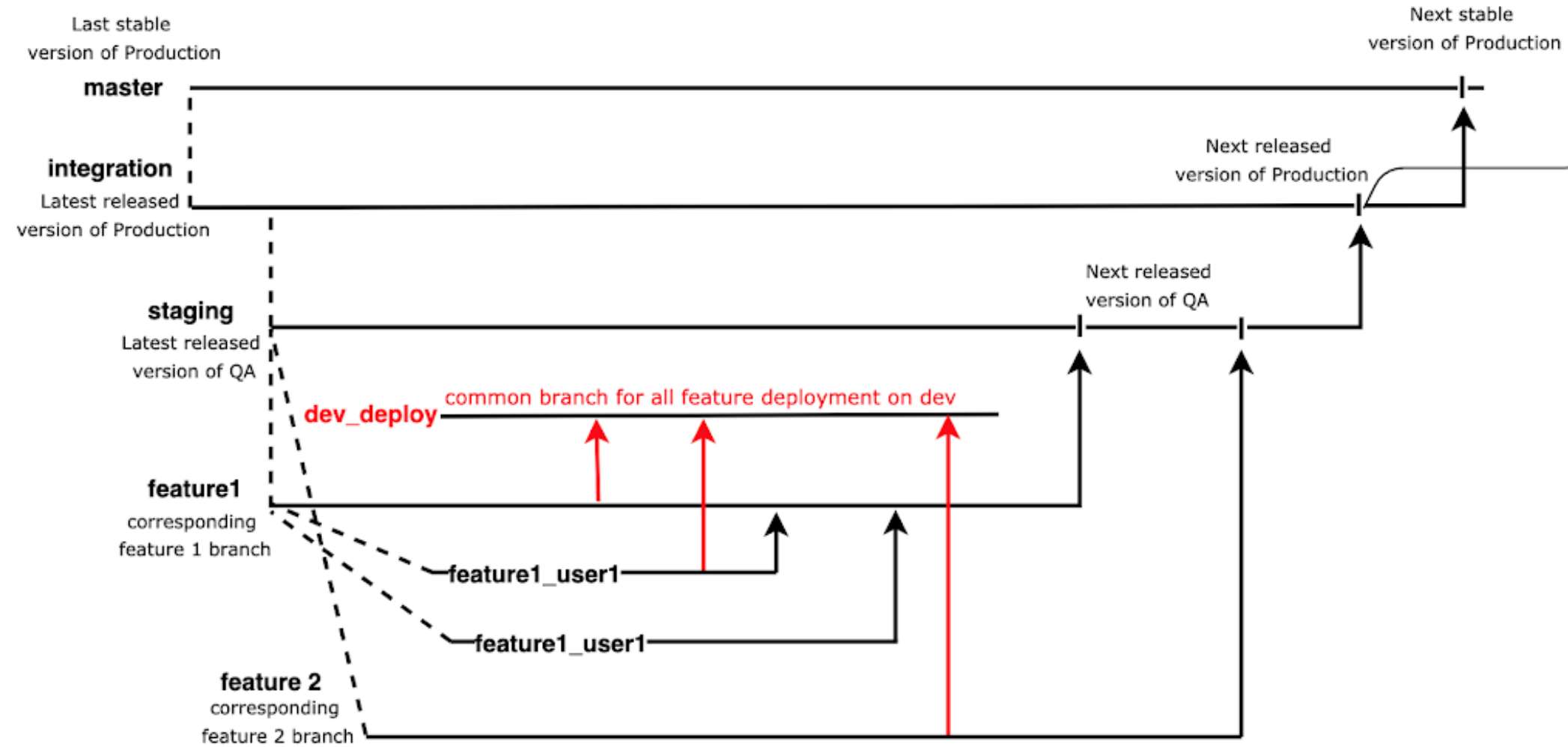
Industry Practices



<https://www.engati.ai/blog/git-branching-strategies>



<https://www.engati.ai/blog/git-branching-strategies>



Deployment day 😎

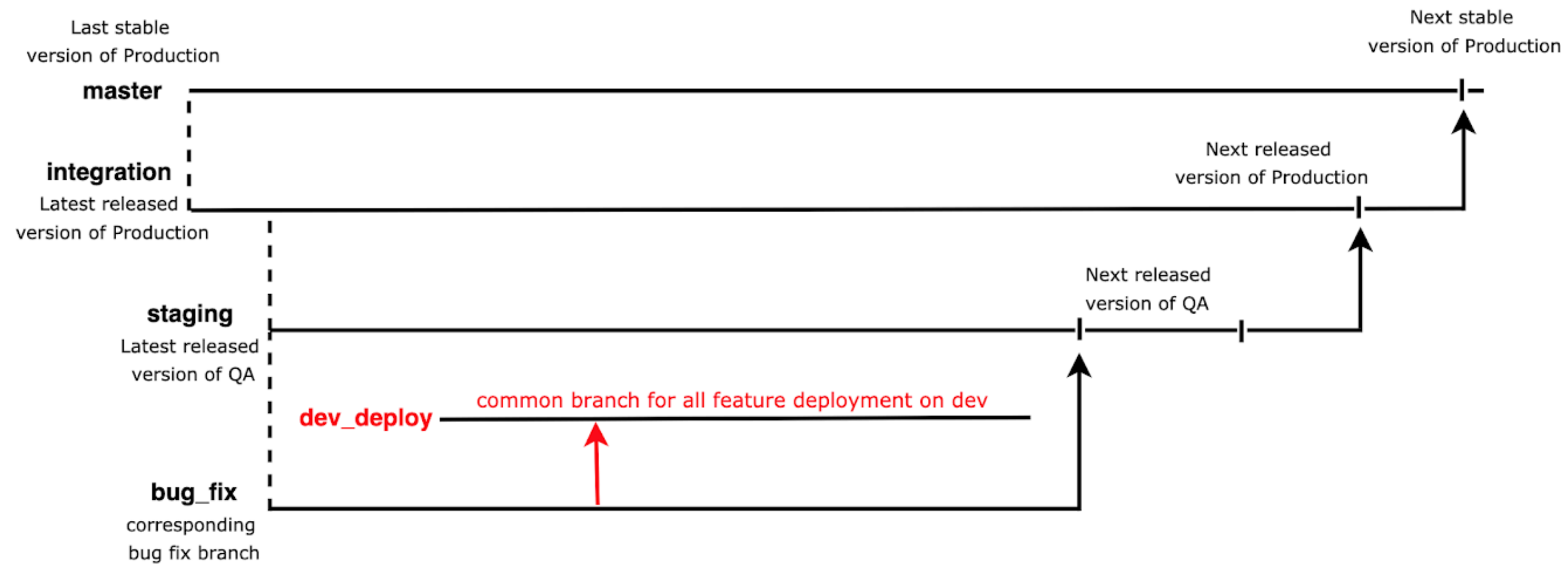
Before merging staging to integration for a big feature release, we will prefer to merge integration to master to make revert easily if needed.

1. Prod deployment will happen once staging merge into integration.

2. After prod deployment in a day or two once we sure that deployment is stable then we will merge integration branch to master.

Branching for feature development

<https://www.engati.ai/blog/git-branching-strategies>



Branching for bug fix

<https://www.engati.ai/blog/git-branching-strategies>

3. Github commands you should master

Core Concepts

Repository:

A folder tracked by Git (your project directory).

Initialized using:

```
git init
```

Commit:

A saved snapshot of your project at a point in time.

Created after staging files:

```
git commit -m "Your message"
```

Branch:

A separate line of development (e.g., for new features or fixes).

Create a new branch:

```
git branch new-feature
```

Remote:

A version of your repository hosted online (like GitHub).

Add a remote repository:

```
git remote add origin https://github.com/username/repo.git
```

HEAD:

A pointer to the latest commit in the current branch.

View where HEAD points:

```
git log --oneline
```

Staging Area:

A preparation zone where you select what to commit.

Add file or files (using just a .) to staging:

```
git add filename.py # single file
```

```
git add . # all the uncommitted files
```

Initializing git repository

Initialize a git repository

git init # → Creates .git folder

Create a sample file

echo "# My First Project" > README.md

Check current status

git status

Add file to staging

git add README.md

Commit changes with message

git commit -m "Initial commit"

Create and Push to GitHub

Set your GitHub repo URL (replace with yours)

```
git remote add origin https://github.com/yourusername/yourrepo.git
```

View connected remotes

```
git remote -v
```

Push to GitHub and track upstream

```
git push -u origin main
```


Branching & Merging

Create a new branch

git branch feature-1

Switch to new branch

git checkout feature-1

or (newer command)

git switch feature-1

Make some changes

echo "New Feature Here" > feature.txt

git add feature.txt

git commit -m "Add feature.txt"

Switch back to main

git checkout main

Merge feature into main

git merge feature-1

Delete the feature branch

git branch -d feature-1

Undoing Changes

Unstage a file

git reset README.md

Restore file to last commit

git restore README.md

Revert a commit (replace with real commit hash)

git revert <commit_hash>

Pulling and Fetching

```
# Get latest changes without merging  
git fetch origin
```

```
# Pull latest from GitHub main branch  
git pull origin main
```

Good Practices

```
# Add a .gitignore
echo "__pycache__/" > .gitignore
echo "env/" >> .gitignore
git add .gitignore
git commit -m "Add .gitignore"

# Update README
echo "This project demonstrates Git basics." >> README.md
git commit -am "Update README"
```

View Log / Help / Info

Show commit history in a compact format with branching graph

git log --oneline --graph

→ Helps visualize branches, merges, and commit history in one line per commit.

Show the current status of the working directory and staging area

git status

→ Tells you which files are modified, staged, or untracked.

List all local branches (and highlight the current one)

git branch

→ Shows your active branch and any other branches created locally.

Show the URLs of remote repositories linked to this project

git remote -v

→ Lists remote sources (e.g., GitHub) for fetch and push operations.

Display help information about Git and its commands

git help

→ Opens Git's built-in manual to explore any command's usage (e.g., 'git help commit').

4. Views

- a. Function based views
- b. Class based views

WHAT IS A **VIEW**?

A Django view is simply a Python function (or class) that takes a request and returns a response.

Analogy:

- Imagine a traffic officer at an intersection.
- Cars (requests) come in.
- The officer checks the situation and directs them.
- Similarly, a view looks at the incoming HTTP request, decides how to handle it, and produces the outgoing HTTP response.

A view can:

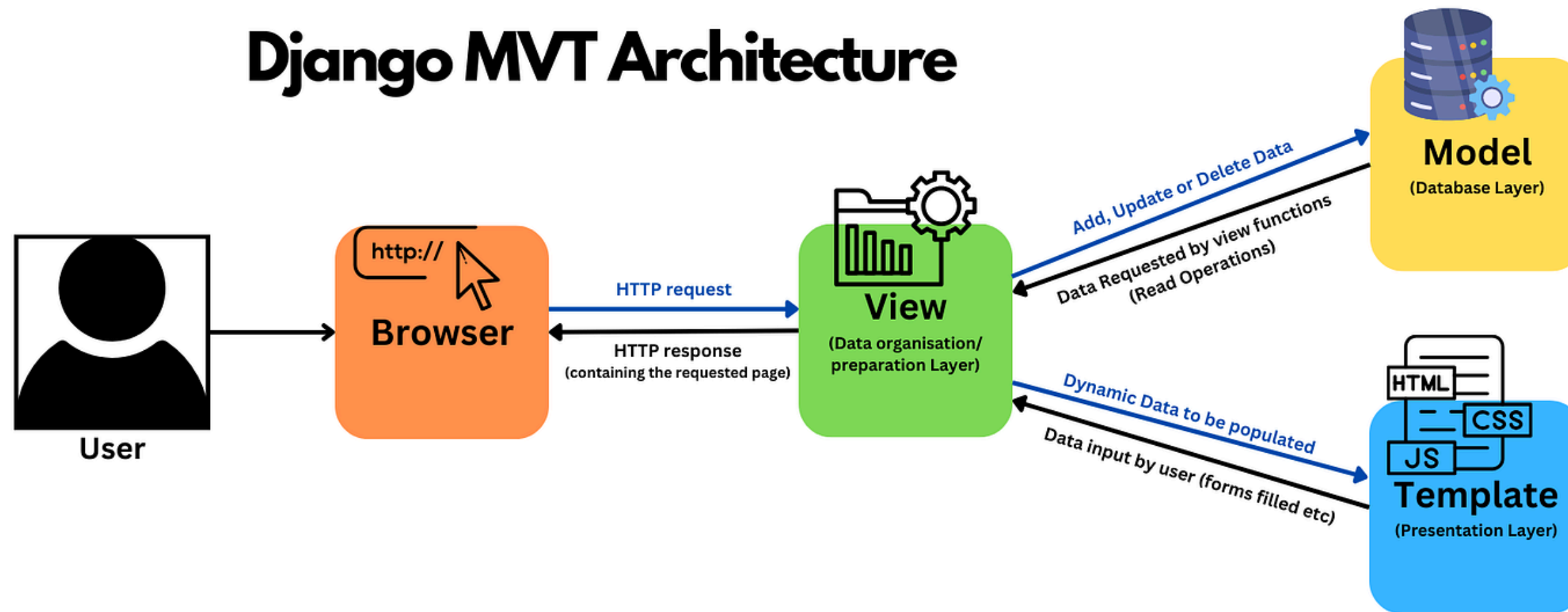
- **Return plain text (HttpResponse).**
- **Render a template (HTML page).**
- **Return JSON data for APIs.**

THINK OF **VIEW** AS THE TRAFFIC CONTROLLER:

- A request comes in (e.g., user visits ***website.com/students/***).
- The view decides what to do:
 - Fetch rows from the Student table.
 - Pass them into a template.
- The view then returns a response back to the browser.

Without views, models and templates **never 'talk' to each other**.

Django MVT Architecture



https://medium.com/@AyushAgrawal_/understanding-django-mvt-architecture-and-view-functions-django-full-course-for-beginners-lesson-39c8da093b44

In how many ways
we can
return html:

2 ways

1. MANUAL WAY

The manual way (using loader.get_template + HttpResponse):

○

```
from django.http import HttpResponse
from django.template import loader

def student_list(request):
    students = Student.objects.all()
    template = loader.get_template("students/student_list.html")
    context = {"students": students}
    output = template.render(context, request)
    return HttpResponse(output)
```

Good for teaching the team/understanding what's really happening under the hood.

2. SHORTCUT WAY: **RENDERING**

In Django, `render()` is a shortcut function that combines three important steps in just two lines:

1. Loads the template

- Django looks for the file `students/student_list.html`.

2. Fills it with context data

- The dictionary `{"students": students}` gets passed into the template.
- Inside the template, you can use `{{ students }}` or loop through them.

```
from django.shortcuts import render

def student_list(request):
    students = Student.objects.all()
    return render(request, "students/student_list.html", {"students": students})
```

3. Returns an `HttpResponse`

- After filling in the template, Django turns it into plain HTML text.
- That HTML is wrapped inside an `HttpResponse` and sent back to the browser.

HttpResponse() vs Render()

```
from django.http import HttpResponse
from django.template import loader

def student_list(request):
    students = Student.objects.all()
    template = loader.get_template("students/student_list.html")
    context = {"students": students}
    output = template.render(context, request)
    return HttpResponse(output)
```

- Django says: You're almost always going to do those three steps together... let's make a **helper function**.
- render(**request**, **template_name**, **context**) combines all 3 steps.

Less typing, cleaner code, easier to read.

You'll use this in real world.

```
from django.shortcuts import render

def student_list(request):
    students = Student.objects.all()
    return render(request, "students/student_list.html", {"students": students})
```



```
views.py x
1 # paste in: students/views.py
2
3 # =====
4 > import ...
12
13 def home(request): 2 usages (1 dynamic)
14     return HttpResponse("Hello from the students app!")
15
16 def student_list_view(request): 4 usages
17
18     student_list_query = Student.objects.all()
19     context = {'student_list_html': student_list_query}
20
21     template = loader.get_template('students/student_list.html')
22
23     output = template.render(context)
24
25     return HttpResponse(output)
```

```
4_8.py x
1 # paste this in students/views.py
2
3 from django.shortcuts import render
4 from .models import Student
5
6 def student_list_view(request):
7
8     student_list_query = Student.objects.all()
9     context = {'student_list_html': student_list_query}
10
11     return render(request, template_name: 'students/student_list.html', context)
```

These were our
Function Based Views

HTTPRESPONSE VS RENDER (SUMMARY)

1. HttpResponse

- **What it is:** The most basic way to send something back to the browser.
- It doesn't care about templates or context. It just wraps any string (HTML, JSON, plain text, etc.) and sends it as the response.
- Use it for testing, or sending plain text/JSON.

Analogy

- **HttpResponse = mailing a handwritten letter.**
- **render = using a predesigned card template and just filling in the names and message before sending.**

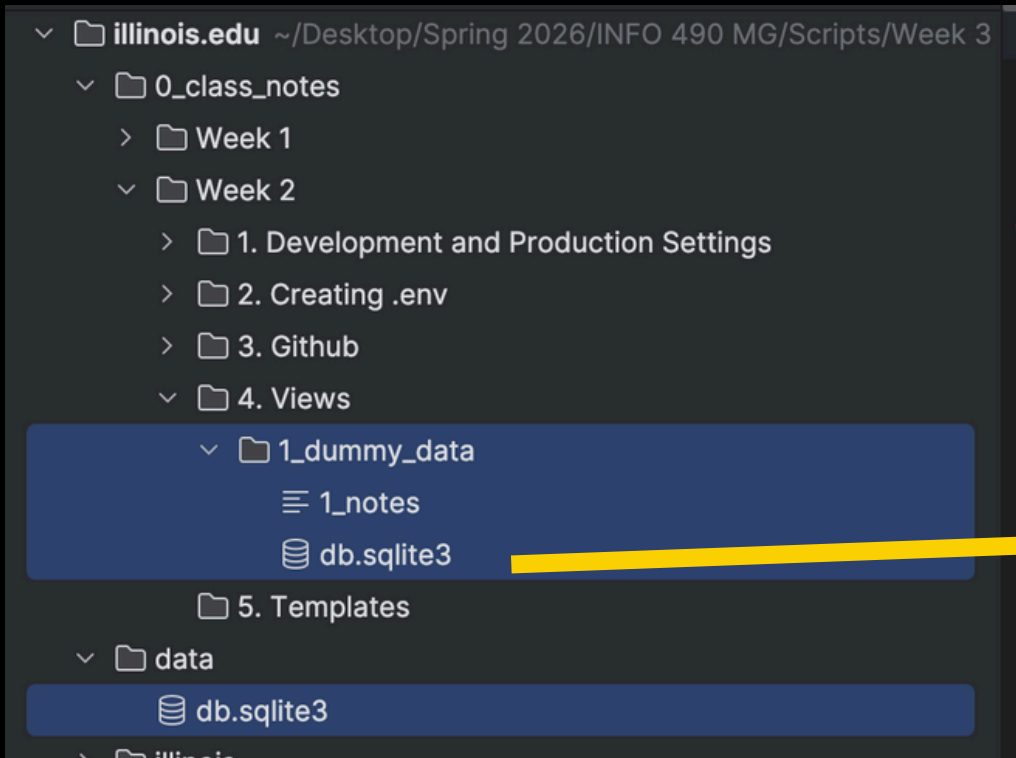
2. Render

- **What it is:** A shortcut that automates three steps you'd otherwise do manually:
 - a. Load an HTML template (.html file).
 - b. Fill it with dynamic data (context dictionary).
 - c. Wrap the final HTML string in an HttpResponse.
- Use it for real web pages, as it ties together views + templates.

Let's Code!

0.
Replace `db.sqlite3`
to the one with
dummy data

New dummy data is now in the database and visible in admin panel



Django administration

WELCOME, MOHITG2. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Home > Students > Enrollments

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

- Groups + Add
- Users + Add

STUDENTS

- Enrollments + Add
- Sections + Add
- Students + Add

Select enrollment to change

Search

Action: ----- Go 0 of 27 selected

ENROLL ID	STUDENT	SECTION	IS ACTIVE	ENROLLED ON
☐ 27	Reed, Ava	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 26	Mitchell, Zane	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 25	Cruz, Yara	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 24	Bennett, Xavier	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 23	Foster, Wendy	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 22	Murphy, Victor	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 21	Sanders, Uma	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 20	Hughes, Tina	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 19	Perez, Sam	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 18	Gonzalez, Rita	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 17	Lopez, Quinn	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 16	Rodriguez, Paul	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 15	Davis, Olivia (Liv)	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 14	Clark, Noah	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 13	Adams, Maya	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 12	Martinez, Leo	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 11	Wilson, Kara	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 10	Brown, Jack	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 9	Wong, Ivy	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 8	Nguyen, Hank	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 7	Hernandez, Grace (Gracie)	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 6	Garcia, Frank	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 5	Patel, Eve (Evie)	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 4	Kim, David	INFO-590-MG (Fall 2026)	✓	Oct. 13, 2025
☐ 3	Lee, Carol	INFO-490-MG (Spring 2026)	✓	Sept. 16, 2025
☐ 2	Smith, Bob	INFO-390-MG (Fall 2025)	✓	Sept. 16, 2025
☐ 1	Johnson, Alice (AJ)	INFO-390-MG (Fall 2025)	✓	Sept. 16, 2025

27 enrollments

FILTER

- Show counts
- By section
 - All
 - INFO-390-MG (Fall 2025)
 - INFO-490-MG (Spring 2026)
 - INFO-590-MG (Fall 2026)
- By is active
 - All
 - Yes
 - No

[ADD ENROLLMENT](#)

ACTIVITY

1. Create a normal .py or .ipnyb file.
2. Now, create a dictionary like in the figure in the file you just created.

```
# Example dictionary for testing
context = {
    "students": [
        {"name": "Alice Johnson", "email": "alice@example.com"},
        {"name": "Bob Smith", "email": "bob@example.com"},
        {"name": "Charlie Kim", "email": "charlie@example.com"},
    ]
}
```

3. Now print the elements of this dictionary such that the output looks like this:

```
Alice Johnson (alice@example.com)
Bob Smith (bob@example.com)
Charlie Kim (charlie@example.com)
```

ANSWER

```
# Define the data (like the Django context)
students = [
    {"name": "Alice Johnson", "email": "alice@example.com"},
    {"name": "Bob Smith", "email": "bob@example.com"},
    {"name": "Charlie Kim", "email": "charlie@example.com"},
]

# Loop through and print each student's info
for s in students:
    print(f"{s['name']} ({s['email']})")
```


How would you perform
similar
operation in
django (Python + HTML)?

ANSWER (HTML SIDE)

```
<h2>Student List</h2>
<ul>
  {% for s in students %}
    <li>{{ s.name }} ({{ s.email }})</li>
  {% empty %}
    <li>No students found.</li>
  {% endfor %}
</ul>
```

Just the way you loop in python, you can do it in HTML

```
# Define the data (like the Django context)
students = [
    {"name": "Alice Johnson", "email": "alice@example.com"},
    {"name": "Bob Smith", "email": "bob@example.com"},
    {"name": "Charlie Kim", "email": "charlie@example.com"},
]

# Loop through and print each student's info
for s in students:
    print(f"{s['name']} ({s['email']})")
```



```
<h2>Student List</h2>
<ul>
    {% for s in students %}
        <li>{{ s.name }} ({{ s.email }})</li>
    {% empty %}
        <li>No students found.</li>
    {% endfor %}
</ul>
```


The method of sending data to HTML templates

```
from django.shortcuts import render

def student_list(request):
    students = Student.objects.all()
    return render(request, "students/student_list.html", {"students": students})
```

Python based
Views

```
<h2>Student List</h2>
<ul>
    {% for s in students %}
        <li>{{ s.name }} ({{ s.email }})</li>
    {% empty %}
        <li>No students found.</li>
    {% endfor %}
</ul>
```

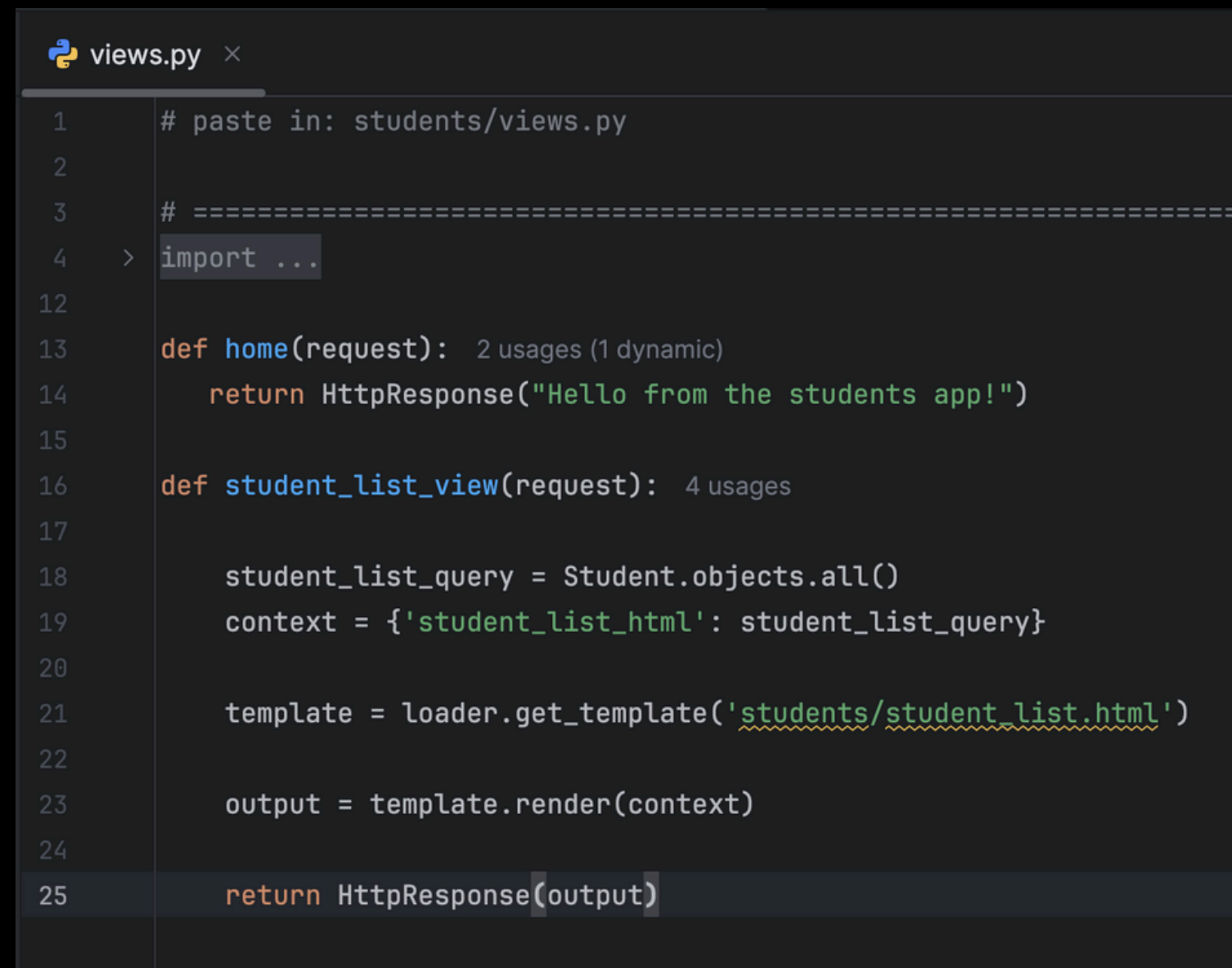
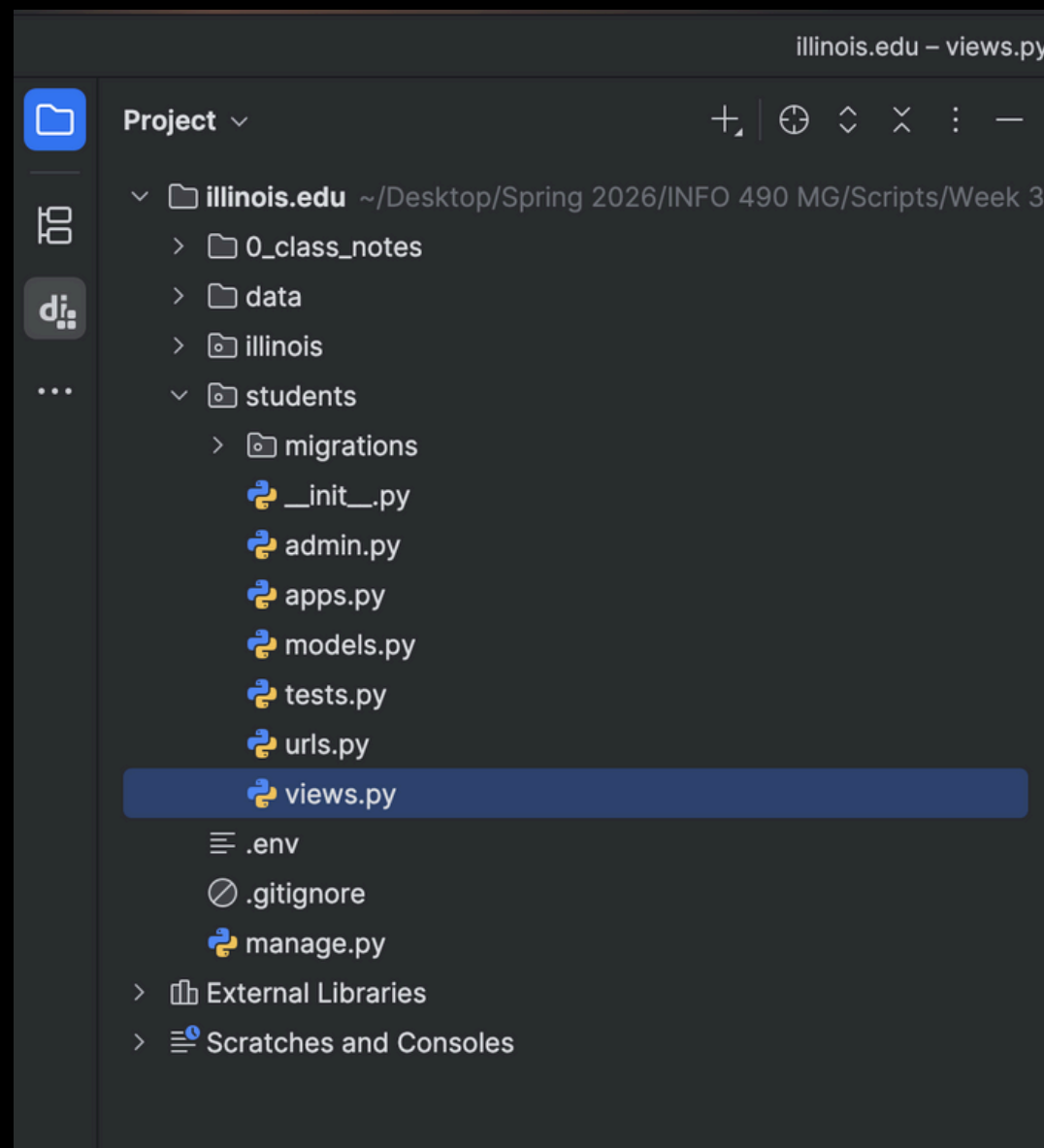
HTML
Template

BACK
TO
PROJECT

Update the students/views.py

Now, the files I gave you are having detailed comments.

Let's read those comments together and understand what's happening.



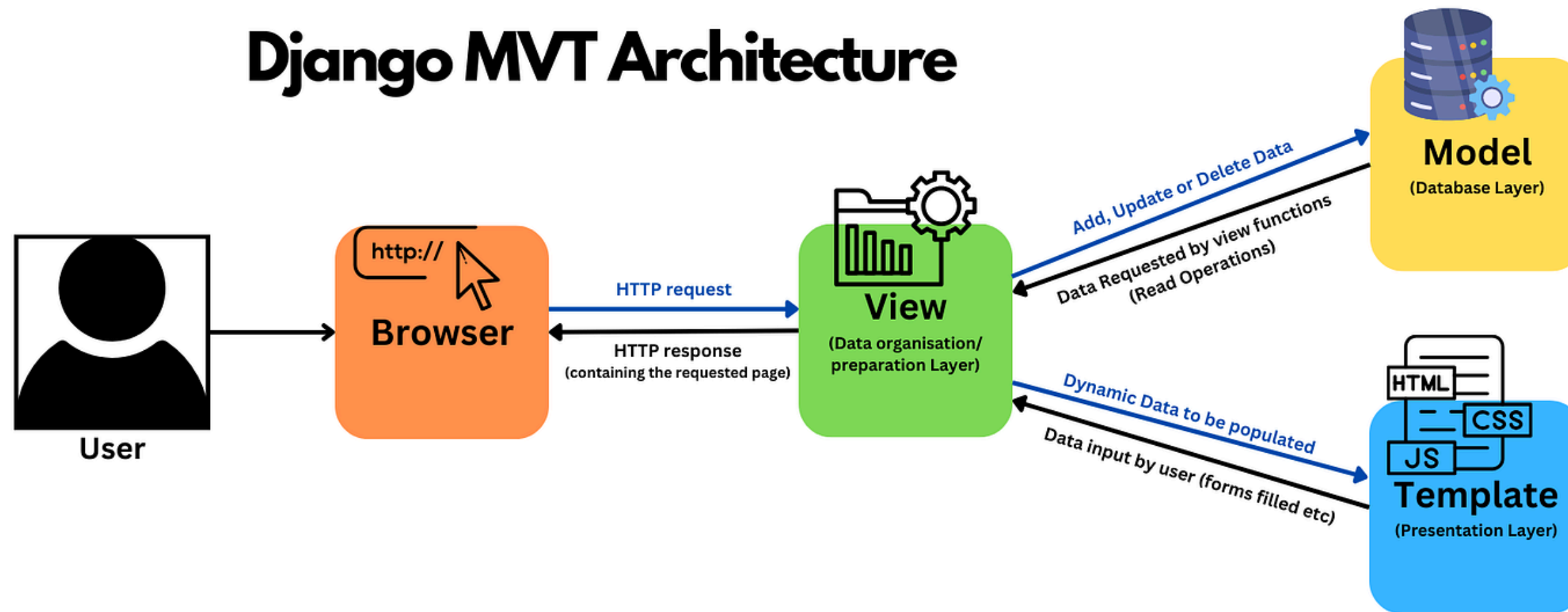
Function based Views

We will now come back to this section for class based views after completing section:

5. Templates

5. Templates

Django MVT Architecture



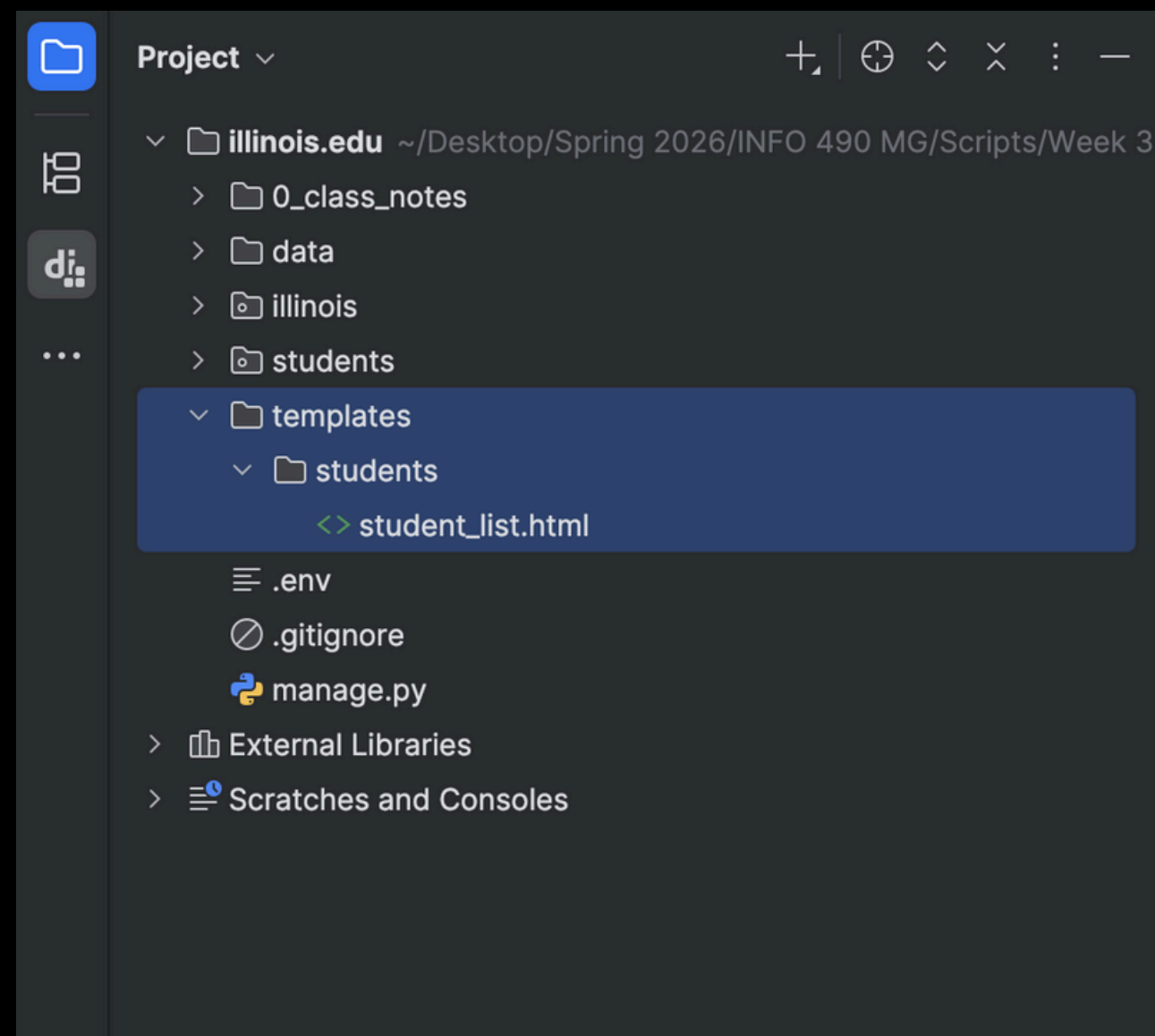
https://medium.com/@AyushAgrawal_/understanding-django-mvt-architecture-and-view-functions-django-full-course-for-beginners-lesson-39c8da093b44

1.
Create
Template Folder

&

Create an HTML file

- Create a directory called **'templates.'**
- In it, create sub-directories with names of all the features or applications that you're building.
- Now, in the students directory, paste file 4_3.html and rename it as student_list.html



```

views.py  student_list.html x
1  <!-- Paste code here: illinois.edu/templates/students/student_list.html -->
2
3  <h2>Student List</h2>
4  <ul>
5      {% for student_for_loop in student_list_html %}
6          <li>
7              <a href="">{{ student_for_loop }}</a>
8          </li>
9
10     {% empty %}
11         <li>No students yet!</li>
12
13     {% endfor %}
14 </ul>

```


- Important: **base.py (master settings)**

- Depending upon where you save your templates folder, **you may run into error.**
- In such scenarios, Django is not able to find your HTML file.
- So you need to make changes in your master settings.py file, and **specify location of your “templates” folder.**

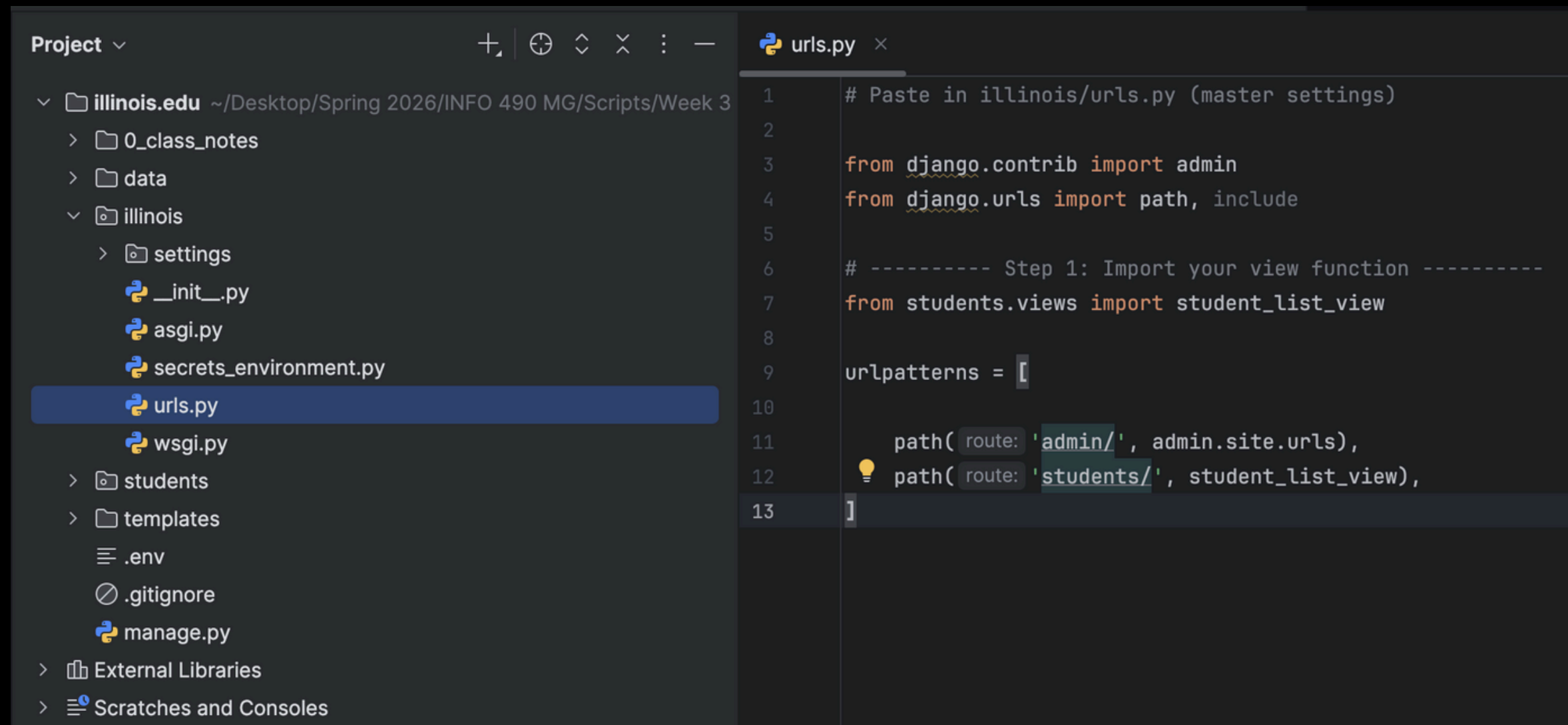
```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
  
        # When you create a common templates folder to be shared for all the apps, use the below approach.  
        'DIRS': [BASE_DIR / 'templates'],  
  
        # In case you're templates folder in every application, comment the above version and use the below:  
        # 'DIRS': [],  
  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    ],  
]
```

2. Create **the URL Patterns**

Now, putting the last piece of puzzle together (file 4_5.py)

View → sends generated data to →

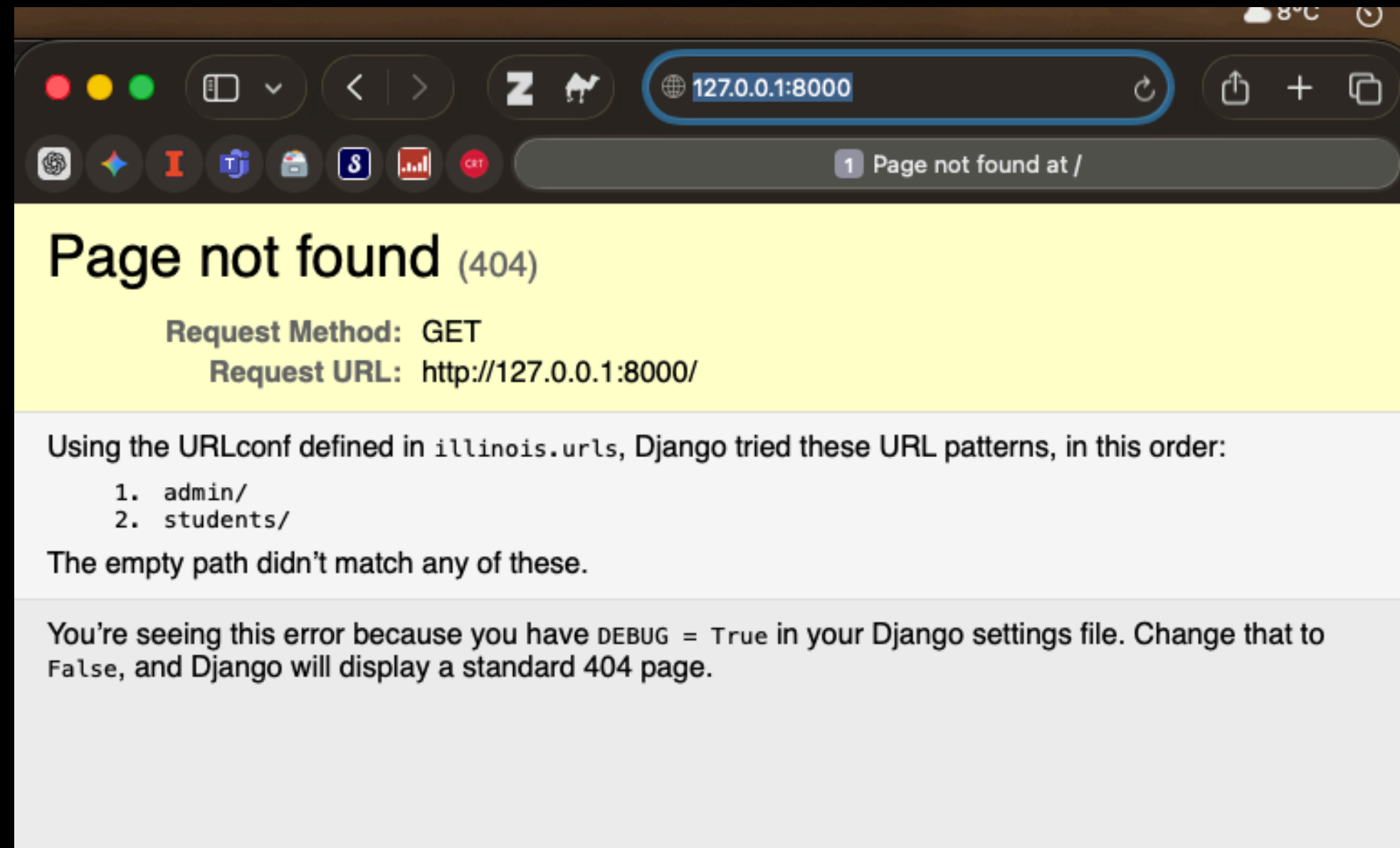
Template → will show user the data upon visiting → **URL**



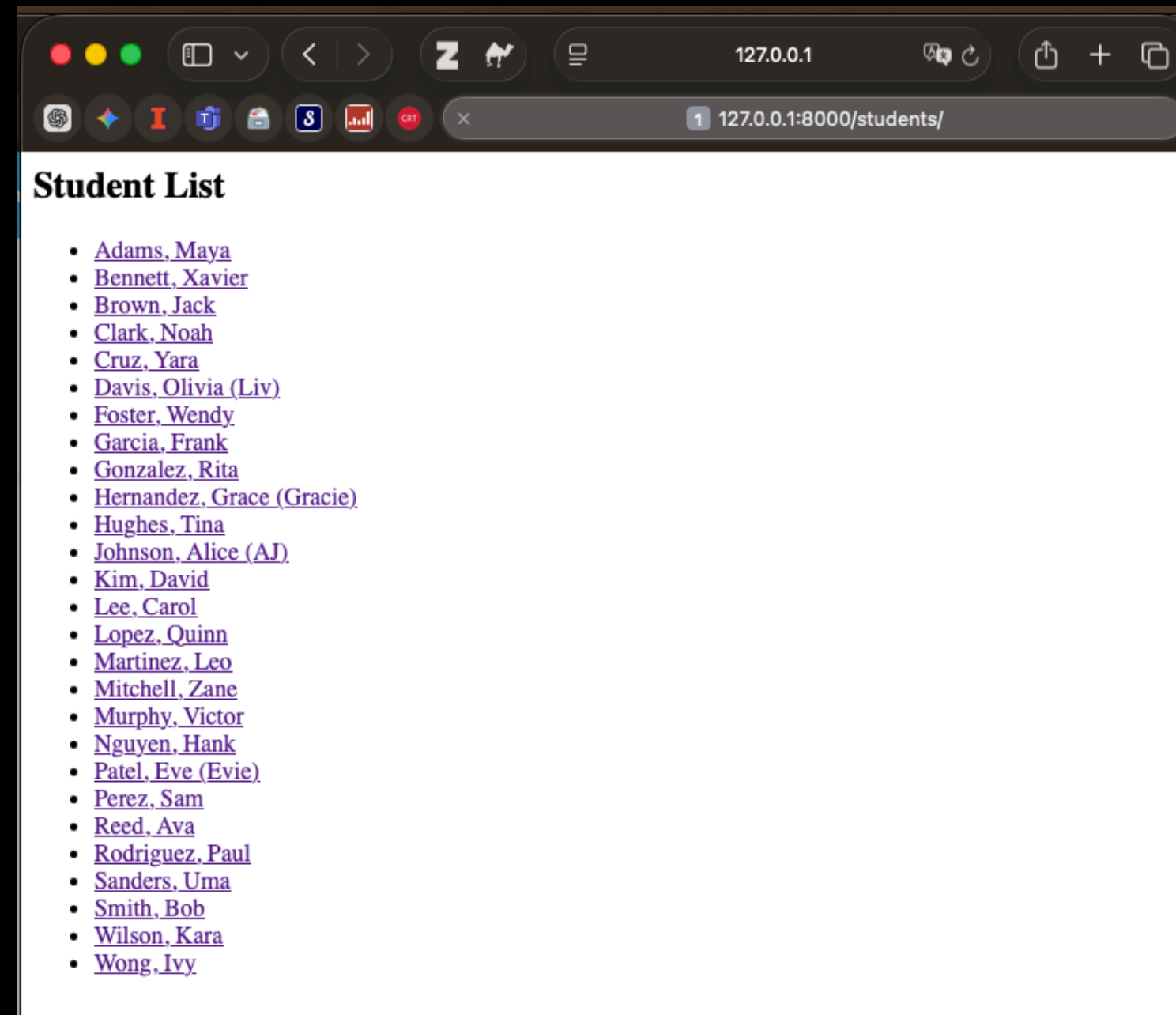
The screenshot shows a code editor with a project explorer on the left and a code editor on the right. The project explorer shows a project named 'illinois.edu' with a file tree. The file 'urls.py' is selected. The code editor shows the following code:

```
1 # Paste in illinois/urls.py (master settings)
2
3 from django.contrib import admin
4 from django.urls import path, include
5
6 # ----- Step 1: Import your view function -----
7 from students.views import student_list_view
8
9 urlpatterns = [
10
11     path(route: 'admin/', admin.site.urls),
12     path(route: 'students/', student_list_view),
13 ]
```

Not an error.
This is debug = True mode.



You should now be able to see the list of students via the url: **127.0.0.1:8000/students**



CREATING A BASE TEMPLATE

- We will put the HTML code that we would want to write once and inherit in multiple files.
- This prevents repeated programming tasks.

Project

illinois.edu ~/Desktop/Spring 2026/INFO 490 MG/Scripts/Week 3

0_class_notes

data

db.sqlite3

illinois

students

templates

students

base.html

student_list.html

.env

.gitignore

manage.py

External Libraries

Scratches and Consoles

base.html x student_list.html

4<html lang="en">

15<body class="bg-light">

26<nav class="navbar navbar-expand-lg navbar-light bg-white border-bottom mb-4">

34</nav>

35

36<!-- Main Content -->

37<main class="container mb-5">

38

39<!-- This should be the name of the block that will show it's content here. -->

40{% block student_content %}

41

42<p class="text-muted">This is default content. </p>

43

44{% endblock %}

45</main>

46

47<!-- Footer -->

48<footer class="bg-dark text-white py-3">

49<div class="container text-center small">

50<p class="mb-1">illinois.edu class demonstration</p>

51<p class="mb-0">

52Licensed under a

53

54Creative Commons Attribution-ShareAlike 4.0 International License

55

56</p>

57</div>

58</footer>

59

60<!-- Optional Bootstrap JavaScript -->

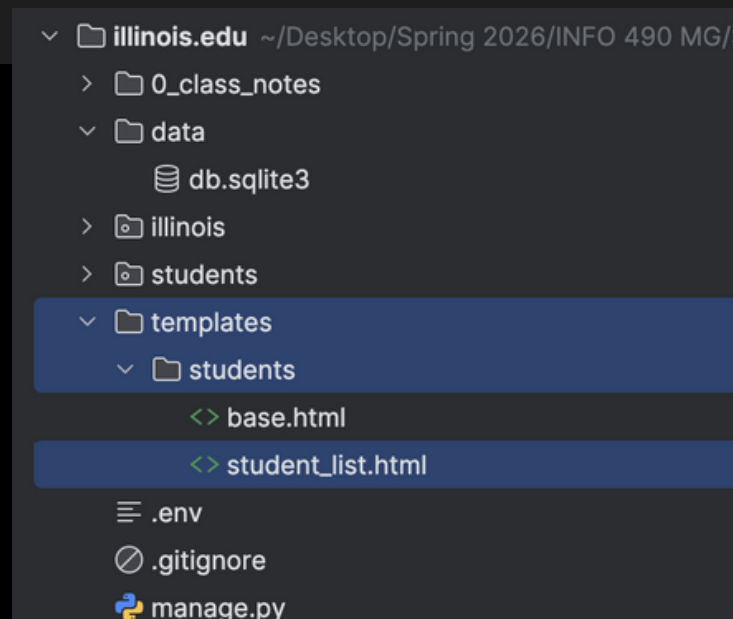
61<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>

62</body>

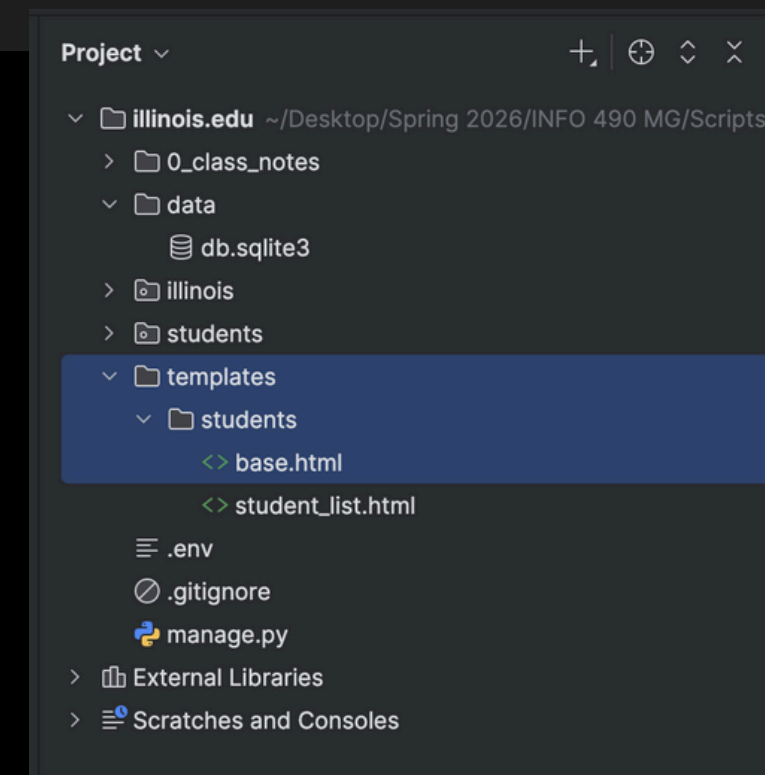
63</html>

Django Structure


```
<> student_list.html x
1  <!-- Paste code here: templates/students/student_list.html -->
2
3  <!-- Step 0: Paste the output of this code inside the base.html -->
4  {% extends "students/base.html" %}
5
6  {% block title %}
7      Student List
8  {% endblock %}
9  💡
10 {% block student_content %}
11     <h2>Student List</h2>
12
13     <ul>
14         {% for student_for_loop in student_list_html %}
15
16             <li>
17                 <a href=""> {{ student_for_loop }}</a>
18             </li>
19
20             {% empty %}
21                 <li>No students yet!</li>
22             {% endfor %}
23
24     </ul>
25 {% endblock %}
```



```
<> student_list.html  <> base.html x
4      <html lang="en">
15     <body class="bg-light">
26         <nav class="navbar navbar-expand-lg navbar-light bg-white">
34         </nav>
35
36         <!-- Main Content -->
37         <main class="container mb-5">
38
39             <!-- This should be the name of the block that will show the content -->
40             {% block student_content %}
41                 <p class="text-muted">This is default content. </p>
42
43             {% endblock %}
44         </main>
45     </body>
46 </html>
```



Cosmetic changes now visible.

illinois.edu
Course Information System

Sections Students Registrations

Student List

- [Adams, Maya](#)
- [Bennett, Xavier](#)
- [Brown, Jack](#)
- [Clark, Noah](#)
- [Cruz, Yara](#)
- [Davis, Olivia \(Liv\)](#)
- [Foster, Wendy](#)
- [Garcia, Frank](#)
- [Gonzalez, Rita](#)
- [Hernandez, Grace \(Gracie\)](#)
- [Hughes, Tina](#)
- [Johnson, Alice \(AJ\)](#)
- [Kim, David](#)
- [Lee, Carol](#)
- [Lopez, Quinn](#)
- [Martinez, Leo](#)
- [Mitchell, Zane](#)
- [Murphy, Victor](#)
- [Nguyen, Hank](#)
- [Patel, Eve \(Evie\)](#)
- [Perez, Sam](#)
- [Reed, Ava](#)
- [Rodriguez, Paul](#)
- [Sanders, Uma](#)
- [Smith, Bob](#)
- [Wilson, Kara](#)
- [Wong, Ivy](#)

illinois.edu class demonstration
Licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](#)

4.2 Class Based Views

(continuation of section 4)

REFACTORING

It means **restructuring or improving the existing codebase / structure without changing its external behavior**. And that is what we did at the beginning of this lecture.

But why **refactor** things in Django?

- **Readability**: Make views, models, and templates easier to understand.
- **Maintainability**: Simplify updates, debugging, and collaboration.
- **Performance**: Reduce redundant queries, improve response times.
- **Reusability**: Extract common logic into reusable functions, classes, or Django apps.
- **Best Practices**: Align code with Django's 'DRY' (Don't Repeat Yourself) principle.

Common Refactoring Practices in Django

1.1 Refactoring URLs

- Use namespaces in urls.py for modular apps.
- Replace deeply nested URLs with clean REST-like paths.

1.2. Refactoring Views

- Replace **function-based views (FBVs)** with **class-based views (CBVs)** when appropriate for reusability.
- Use mixins to avoid repeating logic across views.
- Apply Django's generic views (ListView, DetailView, CreateView, etc.) instead of writing boilerplate.

1.3. Refactoring Templates (We just did this)

- Extract repeated HTML into template inheritance ({% extends %} and {% block %}).

1.4. Refactoring Forms

- Use Django's ModelForms instead of manually defining fields when tied to models.
- Extract form validation logic into clean_<field>() methods instead of duplicating checks.

1.5. Refactoring Models

- Break large models into smaller, logical ones.

1.6. Refactoring Database Queries

- Use select_related and prefetch_related to reduce N+1 query problems.
- Move repeated queriesets into custom managers.

1.7. Project-Level Refactoring

- Break large projects into smaller, reusable Django apps.
- Centralize settings using settings/ package (e.g., base.py, dev.py, prod.py).
- Apply environment variables (via django-environ or python-decouple) for secrets/config.

Common Refactoring Practices in Django

1.1 Refactoring URLs

- Use namespaces in `urls.py` for modular apps.
- Replace deeply nested URLs with clean REST-like paths.

Let's refactor urls.py. Open:

1.0_class_notes/ Week 2 / 5. Refactoring / 5_1.py

- In this file, we'll **cut the path for 'students/' URL pattern from illinois/urls.py** and paste it in **students/urls.py**

```
illinois/urls.py × students/urls.py
1 # Paste in illinois/urls.py (master settings)
2
3 from django.contrib import admin
4 from django.urls import path, include
5
6 # ----- Step 1: Import your view function -----
7 from students.views import student_list_view
8
9 urlpatterns = [
10
11     path(route: 'admin/', admin.site.urls),
12     path(route: 'students/', student_list_view),
13 ]
```

```
illinois/urls.py students/urls.py ×
1 # Paste in: students/urls.py
2
3 # So, we are now organizing it and making it more e
4 # This file will contain only the URLs related to a
5
6 # =====
7
8 from django.urls import path, include
9
10 # Step 1: We import the student_list_view() functio
11 from students.views import student_list_view
12
13 # Now, we link that output to the url associated wi
14 urlpatterns = [
15     path(route: 'students/', student_list_view),
16 ]
```


What we desire

website.com/students

illinois/urls.py

(URLs of all the applications)

students/urls.py

(URLs of specific feature called students)

website.com/students-age

website.com/students-gpa

```
illinois/urls.py ×
1 # Paste in illinois/urls.py (master settings)
2
3 # Now, we are importing all the urls from 'student'
4 # which are related to only its features here at
5
6 # =====
7
8 from django.contrib import admin
9 from django.urls import path, include
10
11 urlpatterns = [
12     path(route: 'admin/', admin.site.urls),
13     path(route: '', include('students.urls')),
14 ]
```

```
illinois/urls.py  students/urls.py ×
1 # Paste in: students/urls.py
2
3 # So, we are now organizing it and making it more e
4 # This file will contain only the URLs related to a
5
6 # =====
7
8 from django.urls import path, include
9
10 # Step 1: We import the student_list_view() functio
11 from students.views import student_list_view
12
13 # Now, we link that output to the url associated wi
14 urlpatterns = [
15     path(route: 'students/', student_list_view),
16 ]
```

Let's Refactor Views Now

1.1 Refactoring URLs

- Use namespaces in `urls.py` for modular apps.
- Replace deeply nested URLs with clean REST-like paths.

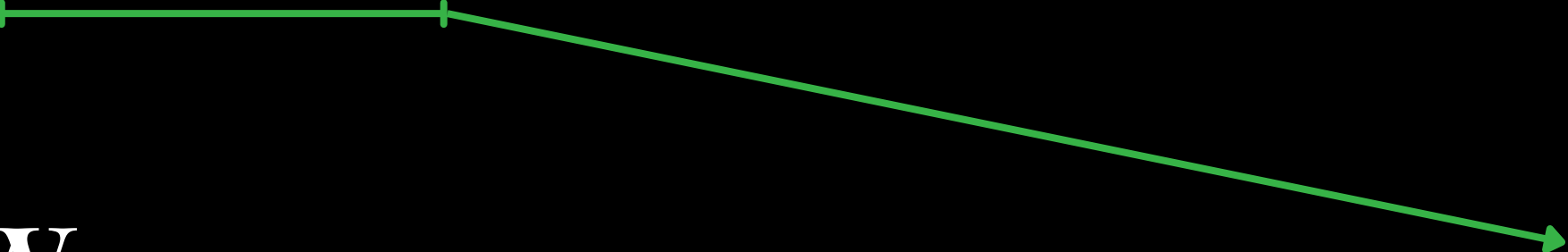
1.2. Refactoring Views

- Replace **function-based views (FBVs)** with **class-based views (CBVs)** when appropriate for reusability.
- Use mixins to avoid repeating logic across views.
- Apply Django's generic views (`ListView`, `DetailView`, `CreateView`, etc.) instead of writing boilerplate.

4.2.1

THE **BASE**

VIEW
CLASS



StudentListView(View**)**

vs

StudentListView(ListView)

1. Using the base View class:

- Here you inherit from View (the most generic/common class).
- **You write the logic yourself inside get().**
- That's why you must explicitly call `Student.objects.all()` in the context.
- View does not know about your model. It just gives you a place to define request-handling methods (get, post, etc.).

Conclusion: You don't use `model = Student` here. It is because the base View class has no idea what to do with it.

```
class StudentListView(View):
    def get(self, request):
        return render(
            request,
            'students/student_list.html',
            context={'student_list_html': Student.objects.all()}
        )
```

4.2.2

THE

GENERIC

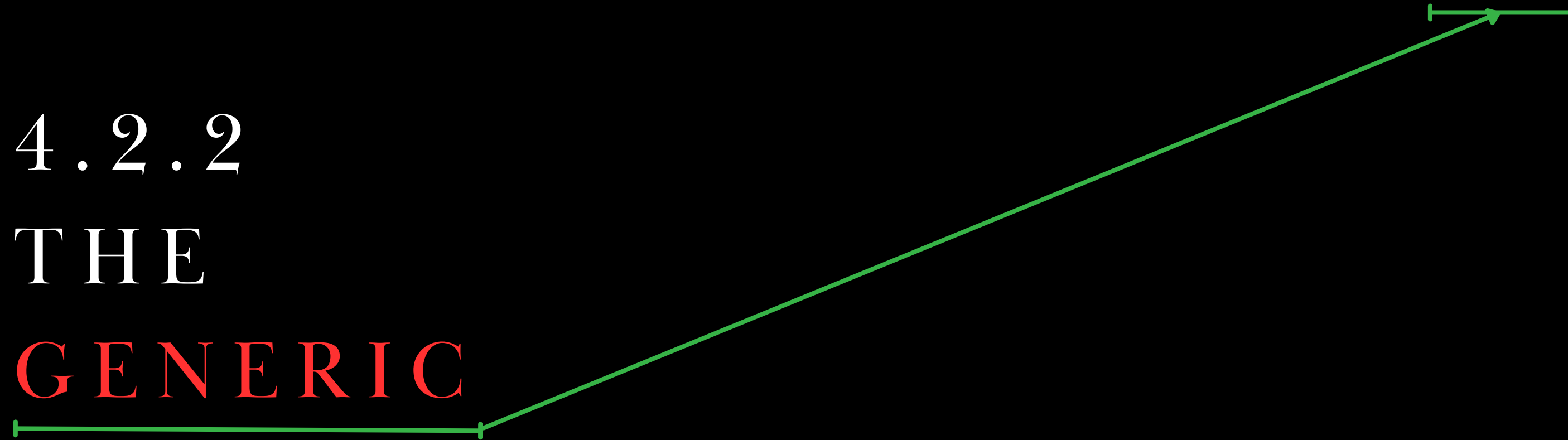
VIEW

CLASS

StudentListView(View)

vs

StudentListView(ListView)



2. Using a Generic Class-Based View (e.g., ListView)

- ListView is a specialized CBV provided by Django for listing objects.
- **When you declare `model = Student`, Django automatically runs `Student.objects.all()` under the hood.**
- You don't need to implement `get()` manually.
- It also adds useful features like pagination and queryset customization.

```
class StudentListView(ListView):  
    model = Student  
    template_name = 'students/student_list.html'  
    context_object_name = 'student_list_html'
```

Conclusion: With `ListView`, the `model` attribute is meaningful and replaces the need to fetch objects manually.

Common Practices for Class Based Views (CBV) in Django

1. Follow the <Model><Action>View naming pattern

- Start with the model name.
- End with the type of view (action).
- Helps developers immediately understand the purpose.



```
class ArticleListView(ListView):  
    ...  
  
class ArticleDetailView(DetailView):  
    ...  
  
class ArticleCreateView(CreateView):  
    ...  
  
class ArticleUpdateView(UpdateView):  
    ...  
  
class ArticleDeleteView>DeleteView):  
    ...
```

2. Use PascalCase (CapWords) for Classes

- Always **capitalize each word**, no underscores.

3. Be Descriptive for Custom Views

- If you extend or customize a CBV, keep names clear.
- If a view doesn't directly map to CRUD (e.g., search, dashboard, report), describe its intent:



```
class ArticleSearchView(ListView):  
    ...  
class UserDashboardView(TemplateView):  
    ...  
class SalesReportView(View):  
    ...
```

4. Use Consistent Template Names

- If you don't explicitly set template_name, Django will try to load a template based on pattern:
 - **<app_name>/<model>_<action>.html**
 - **app_name** = name of the Django app
 - **model_name** = lowercase model name
 - **action** = the type of view (list, detail, form, confirm_delete, etc.)



```
templates/  
  articles/  
    article_list.html  
    article_detail.html  
    article_form.html  
    article_confirm_delete.html
```

Default template name conventions (a few examples):

1. ListView

- **Purpose:** Display a list of objects from the database.
- **Handles:** Querying all objects, pagination, and passing them to a template.
- **Default template:** <app>/<model>_list.html
- **Example use:** Show all blog posts, all users, all products.

```
class ArticleListView(ListView):  
    model = Article
```

2. DetailView

- **Purpose:** Show the details of a single object.
- **Handles:** Fetching an object by primary key (pk) or slug.
- **Default template:** <app>/<model>_detail.html
- **Example use:** A single article page, user profile page, product details.

```
class ArticleDetailView(DetailView):  
    model = Article
```

3. CreateView & UpdateView

- **Purpose:** Display a form for creating/updating a new object and save it.
- **Handles:** Rendering a form, validating input, saving the object.
- **Default template:** <app>/<model>_form.html
- **Example use:** New blog post form, user signup form, product submission.

```
class ArticleCreateView(CreateView):  
    model = Article  
    fields = ['title', 'content']  
    success_url = '/articles/'
```

```
class ArticleUpdateView(UpdateView):  
    model = Article  
    fields = ['title', 'content']  
    success_url = '/articles/'
```

4. DeleteView

- **Purpose:** Confirm and delete an existing object.
- **Handles:** Fetching object, confirmation page, deletion.
- **Default template:** <app>/<model>_confirm_delete.html
- **Example use:** Delete article, remove account, delete product listing.

```
class ArticleDeleteView>DeleteView):  
    model = Article  
    success_url = '/articles/'
```


Overriding Default Template Name Pattern

If you don't want to follow this convention, you can explicitly set the template:

```
class ArticleListView(ListView):  
    model = Article  
    template_name = 'articles/all_articles.html'
```

Why Choose One Over the Other?

4.2.1

THE **BASE**

VIEW
CLASS

StudentListView(**View**)

- You want full control over the request/response cycle.
- The logic is very custom (not a simple CRUD pattern).

```
class StudentListView(View):
    def get(self, request):
        return render(
            request,
            'students/student_list.html',
            context={'student_list_html': Student.objects.all()}
        )
```

4.2.2.

THE **GENERIC**

VIEW
CLASS

StudentListView(**ListView**)

- You're following a common CRUD pattern (list, detail, create, update, delete).
- You want less boilerplate and more Django 'magic' to save time.

```
class StudentListView(ListView):
    model = Student
    template_name = 'students/student_list.html'
    context_object_name = 'student_list_html'
```

But You Can Have More Control over CBVs

1.2.1

```
class StudentListView(ListView):  
    model = Student
```



```
{# using generic name #}  
{% for student_for_loop in object_list %}  
    <li><a href="">{{ student_for_loop }}</a></li>  
{% empty %}  
    <li>No students yet!</li>  
{% endfor %}
```

1.2.2

```
from django.views.generic import ListView  
from .models import Student  
  
class StudentListView(ListView):  
    model = Student  
    template_name = "students/student_list.html"  
    context_object_name = "student_list_html"
```



```
{% for student_for_loop in student_list_html %}  
    <li><a href="">{{ student_for_loop }}</a></li>  
{% empty %}  
    <li>No students yet!</li>  
{% endfor %}
```

Let's reference dynamic url "shortcut names" in the html pages.

This links url patterns in urls.py with HTML

1.

```
# Step 2: define the url
# Check slides of week 5 to see the purpose of 'name'.
urlpatterns = [
    path(route: 'students/',
         StudentList.as_view(),
         name='student-list-url'),
]
```

students/urls.py

```
<a href="{% url 'student-list-url' %}">All Students</a>

<!-- This will generate -->
<a href="/students/">All Students</a>
```

students/student_list.html

2.

```
# Step 2: define the url
# Check slides of week 5 to see the purpose of 'name'.
urlpatterns = [
    path(route: 'students_new_url/',
         StudentList.as_view(),
         name='student-list-url'),
]
```

```
<!-- --- templates/students/student_list.html --- -->
<a href="{% url 'student-list-url' %}">All Students</a>

<!-- This will generate -->
<a href="/students_new_url/">All Students</a>
```

4.2.3

DETAIL

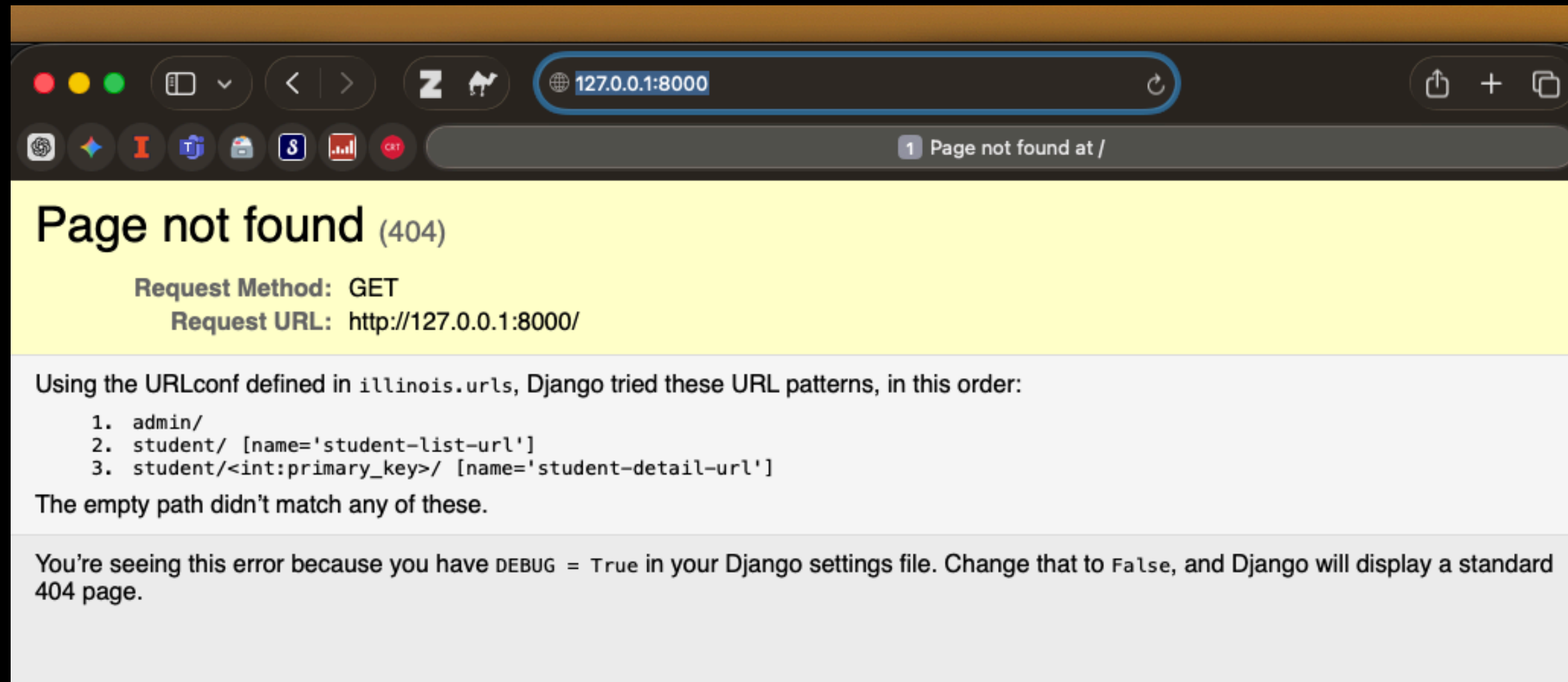
VIEW
CLASS

Let's try few of the examples

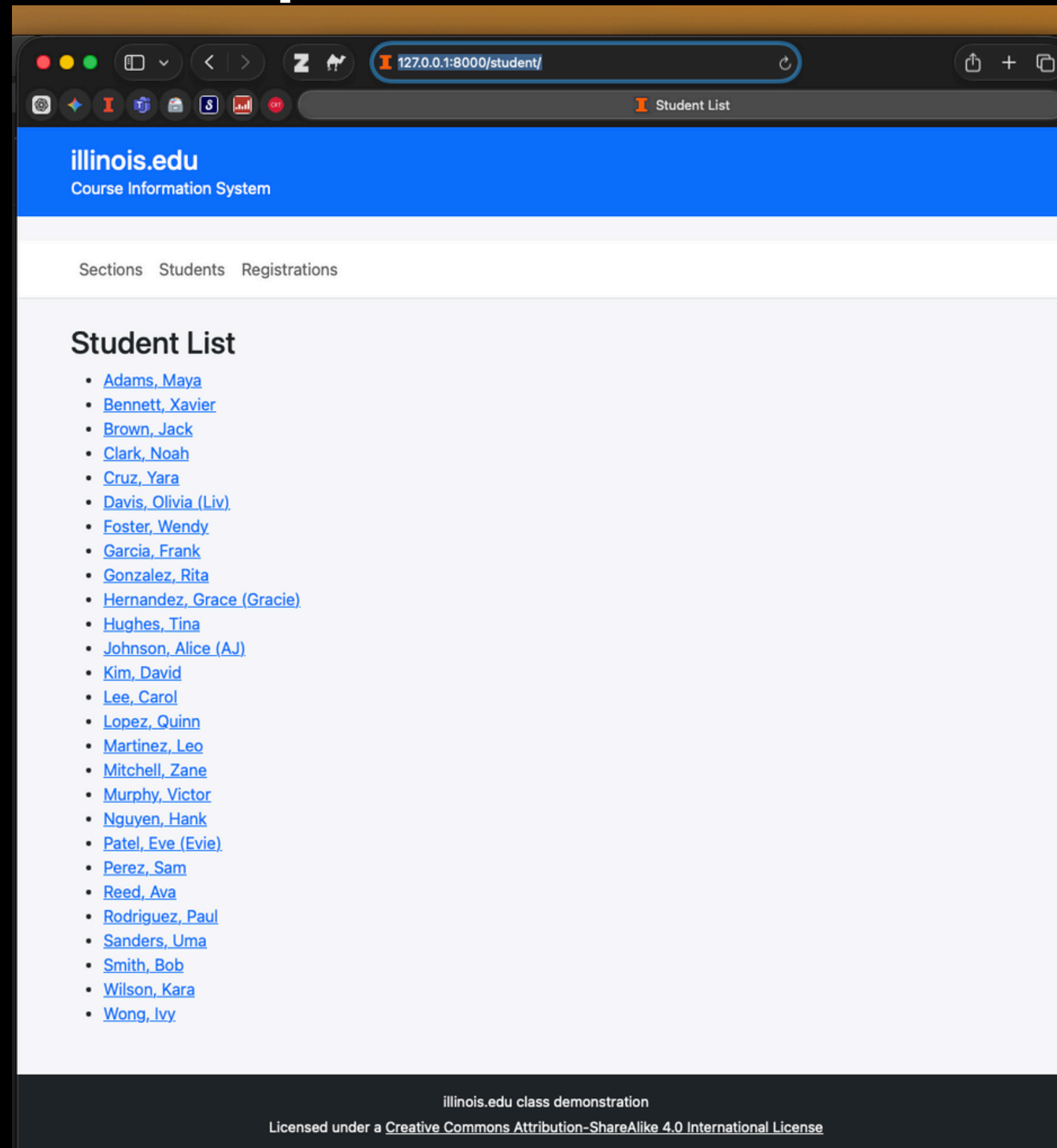
Try following scripts in **“Week 2/ 5. Views & Refactoring”**:

files from 5_3.py till

Home page now shows new links



Final output: student/



Final output: student/1
try more primary keys like: [/2, /3]

The screenshot shows a web browser window with the address bar containing '127.0.0.1:8000/student/1/'. The browser's tab bar shows a single tab titled '1 Student Detail'. The page header is blue with the text 'illinois.edu Course Information System'. Below the header, there are navigation links: 'Sections', 'Students', and 'Registrations'. The main content area displays the student's name 'Johnson, Alice (AJ)' and their details: 'First name: Alice', 'Last name: Johnson', 'Nickname: AJ', and 'Email: alice@example.com'. Below this, there is a section titled 'Enrolled Sections' with a single entry: 'INFO-390-MG — Web App Dev I (Fall 2025)'. The footer of the page is dark gray and contains the text 'illinois.edu class demonstration' and 'Licensed under a Creative Commons Attribution-ShareAlike 4.0 International License'.

illinois.edu
Course Information System

Sections Students Registrations

Johnson, Alice (AJ)
First name: Alice
Last name: Johnson
Nickname: AJ
Email: alice@example.com

Enrolled Sections

- INFO-390-MG — Web App Dev I (Fall 2025)

illinois.edu class demonstration
Licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](#)

THAT'S ALL
FOR TODAY